

**NATIONAL HO CHI MINH UNIVERSITY  
HO CHI MINH UNIVERSITY OF TECHNOLOGY  
ELECTRONIC-ELECTRICAL DEPARTMENT**

-----o0o-----



**GRADUATION THESIS**

**NOVEL MATMUL-FREE COPROCESSOR  
ARCHITECTURE FOR LARGE LANGUAGE  
MODEL**

**INSTRUCTOR: PhD. Trần Hoàng Linh**

**STUDENT: Nguyễn Nhựt Khanh**

**ID: 2151097**

**HO CHI MINH CITY, June 2025**

----- ☆ -----

----- ☆ -----

Số: \_\_\_\_\_/BKĐT  
Khoa: **Điện – Điện tử**  
Bộ Môn: **Điện Tử**

## NHIỆM VỤ LUẬN VĂN TỐT NGHIỆP

1. HỌ VÀ TÊN: Nguyễn Nhật Khanh MSSV: 2151097
  2. NGÀNH: HỆ THÔNG MẠCH - PHẦN CỨNG LỚP : TT21DDT1
  3. Đề tài: Novel Matmul-free Coprocessor Architecture for Large language model
  4. Nhiệm vụ (Yêu cầu về nội dung và số liệu ban đầu):
    - Thiết kế tập lệnh RISC 5 giai đoạn (Fetch, Decode, Execute, Memory, Write Back) hỗ trợ các phép toán vector LDV, STV, ADD, SUB, MUL, DIV, EXP, SIG, NORM, TMATMUL với vector kích thước  $1 \times 512$ .
    - Triển khai RTL các khối chức năng: Register File FIFO ( $8 \times 1$  K-entry), ALU row-wise, TMATMUL 8-stage accumulation và DRAM Router trên FPGA Kintex-7 XC7K325T; mục tiêu tần số 200 MHz.
    - Phân tích kết quả, so sánh với mô hình MatMul truyền thống (thời gian  $\sim 16$  ms/64 tokens, thông lượng  $\sim 62$  tokens/s, công suất  $\sim 13$  W), hoàn thiện báo cáo luận văn.
  5. Ngày giao nhiệm vụ luận văn: 09/01/2025
  6. Ngày hoàn thành nhiệm vụ: 25/04/2025
  7. Họ và tên người hướng dẫn: Phần hướng dẫn  
PhD. Trần Hoàng Linh 100%
- Nội dung và yêu cầu LVTN đã được thông qua Bộ Môn.

*Tp.HCM, ngày..... tháng..... năm 2025*  
**CHỦ NHIỆM BỘ MÔN**

**NGƯỜI HƯỚNG DẪN CHÍNH**

### PHẦN DÀNH CHO KHOA, BỘ MÔN:

Người duyệt (chấm sơ bộ):.....  
Đơn vị:.....  
Ngày bảo vệ : .....  
Điểm tổng kết: .....  
Nơi lưu trữ luận văn: .....

## ***ACKNOWLEDGEMENT***

*First and foremost, I would like to express my deepest gratitude to my advisor, for their invaluable guidance, support, and encouragement throughout the course of this project. Their expertise and insights have been instrumental in shaping the direction of my research and overcoming challenges along the way.*

*I would also like to extend my sincere thanks to my professors and lecturers at Ho Chi Minh University of Technology for providing a solid foundation of knowledge and skills, which has greatly contributed to the completion of this thesis.*

*This thesis would not have been possible without the contributions and encouragement of everyone mentioned above. Thank you all for being part of this journey.*

*Ho Chi Minh City, June 4<sup>th</sup> 2025 .*

**Student**

## OVERVIEW

This project focuses on designing and implementing hardware acceleration for Token Generation of large language models (LLMs) using a RISC-based pipeline structure. The objective is to address the computational and memory challenges associated with traditional matrix multiplication (MatMul) operations in LLMs by leveraging lightweight MatMul-free architectures.

The design integrates a 5-stage RISC pipeline (Fetch, Decode, Execute, Memory, Write Back) to optimize data flow and processing efficiency. Key features include the use of ternary quantization for lightweight operations, DDR3 FIFO for efficient data storage, and a control unit to manage read/write signals and hazard detection. TMATMUL (Ternary Matrix Multiplication) is incorporated into the memory stage, continuously retrieving data for efficient computation.

The system was implemented on FPGA hardware to exploit its parallel processing capabilities, achieving significant improvements in memory usage, latency, and power consumption. The results demonstrate that MatMul-free architectures can provide efficient and scalable solutions for LLM inference tasks, especially in resource-constrained environments.

This research highlights the potential of RISC-based hardware pipelines to enable lightweight, sustainable AI applications and sets the stage for further developments in efficient language modeling systems.

TABLE OF CONTENTS

1. INTRODUCTION ..... 1

1.1 Motivation..... 1

1.2 Background.....2

1.3 Research Gap .....5

1.4 Research Objective .....7

1.4.1. Core Goals .....7

1.4.2. Research Questions.....8

1.4.3. Expected Contributions .....8

1.5 Structure of the Thesis .....9

2. RELATED WORKS.....10

2.1 Scalable MatMul-free Language Modeling.....10

2.2 Quantization and Low-Precision Language Models .....13

2.2.1 Full-Precision vs. Quantized Models.....13

2.2.2 Ternary Quantization .....13

2.2.3 Activation Quantization and RMS Normalization .....14

2.2.4 Ternary LLMs in Literature.....14

2.3 FPGA Acceleration for LLMs .....15

3. THEORY .....17

3.1 Token generation .....17

3.2 The Cost of Matrix Multiplication.....18

|                                                     |    |
|-----------------------------------------------------|----|
| 3.2.1 Mathematical Formulation.....                 | 18 |
| 3.2.2 Hardware Implications.....                    | 19 |
| 3.2.3 Energy and Latency Overhead.....              | 20 |
| 3.2.4 Scalability Challenges .....                  | 20 |
| 2.3.3 Hardware Implications.....                    | 21 |
| 3.3 BitLinear Layers and Ternary Quantization ..... | 22 |
| 3.3.1 Ternary Weight Quantization .....             | 22 |
| 3.3.2 Quantized Activation Processing.....          | 23 |
| 3.3.3 Fused BitLinear Layer .....                   | 24 |
| 3.4 MLGRU: A MatMul-Free Token Mixer .....          | 24 |
| 3.4.1 Motivation.....                               | 25 |
| 3.4.2 Standard GRU Overview .....                   | 25 |
| 3.4.3 MLGRU Formulation .....                       | 26 |
| 3.4.4 Computational Simplification.....             | 26 |
| 3.5 MatMul-Free Channel Mixer: BitLinear-GLU.....   | 27 |
| 4. HARDWARE DESIGN AND IMPLEMENTATION .....         | 30 |
| 4.1 Overview of the MatMul-Free Coprocessor.....    | 30 |
| 4.2 Risc-based custom instruction set.....          | 32 |
| 4.3 Register File and FIFO Integration.....         | 35 |
| 4.3.1 Architecture and Addressing Scheme.....       | 36 |
| 4.3.2 FIFO Read and Write State Machines .....      | 37 |

|                                                              |    |
|--------------------------------------------------------------|----|
| 4.3.3 Pipeline Integration and Hazard Management .....       | 38 |
| 4.3.4 Write-Back and Result Handling.....                    | 38 |
| 4.4 TMATMUL Block.....                                       | 38 |
| 4.4.1 Motivation and Design Goals .....                      | 38 |
| 4.4.2 Block Architecture and Dataflow .....                  | 39 |
| 4.4.3 Internal FSM and Pipelining.....                       | 40 |
| 4.4.4 Parallelism and Performance Optimization .....         | 41 |
| 4.4.5 Control and Integration.....                           | 42 |
| 4.5 Arithmetic Logic Unit (ALU).....                         | 43 |
| 4.5.1 Supported Operations .....                             | 43 |
| 4.5.2 Vectorized Row-Wise Architecture.....                  | 44 |
| 4.5.3 Lookup Table Accelerated Nonlinear Functions.....      | 44 |
| 4.5.5 Normalization via RMS .....                            | 44 |
| 4.6 Memory Subsystem .....                                   | 45 |
| 4.6.1 Overview.....                                          | 45 |
| 4.6.2 Memory Layout and Address Decoding.....                | 45 |
| 4.6.3 Read and Write Finite State Machines .....             | 47 |
| 4.6.4 FIFO-Style Access Control and Pointer Management ..... | 48 |
| 4.6.5 Flow Control and Status Signaling .....                | 48 |
| 4.6.6 Ternary Matrix Multiplication Support .....            | 49 |
| 4.7 Hazard Detection and Pipeline Control .....              | 49 |

|                                                             |    |
|-------------------------------------------------------------|----|
| 4.7.1 Motivation and Overview .....                         | 49 |
| 4.7.2 Types of Hazards in the Coprocessor .....             | 50 |
| 4.7.3 Hazard Resolution and Control Signal Generation ..... | 50 |
| 4.7.4 Integration with Pipeline Registers.....              | 51 |
| 4.7.5 Design Tradeoffs .....                                | 51 |
| 4.8 Detailed Block Diagram .....                            | 52 |
| 5. IMPLEMENTATION RESULT .....                              | 53 |
| 5.1 Hardware.....                                           | 53 |
| 5.2 Synthesis Result.....                                   | 55 |
| 5.3 Simulation Result.....                                  | 56 |
| 6. CONCLUSION AND FUTURE DIRECTIONS .....                   | 63 |
| 6.1 Conclusion .....                                        | 63 |
| 6.2 Future Directions .....                                 | 64 |
| 6.2.1. Performance Optimization and Resource Scaling .....  | 64 |
| 6.2.2. Extending Model Capacity and Flexibility.....        | 64 |
| 6.2.3. Hardware-Software Co-design .....                    | 64 |
| 6.2.4. Application Deployment.....                          | 65 |
| 6.2.5. Comparative Benchmarking and Standardization .....   | 65 |
| 6.3 Closing Remark .....                                    | 65 |
| 7. References.....                                          | 66 |

## LIST OF FIGURES

|                                                                                                            |    |
|------------------------------------------------------------------------------------------------------------|----|
| Figure 1: Comparison of GPU Memory and Latency in inference between MatMul-free LM and Transformer++ ..... | 2  |
| Figure 2: RTL implementation for running MatMul-free token generation .....                                | 16 |
| Figure 3: Visual illustration of ternary quantization and BitLinear layers .....                           | 23 |
| Figure 4: Overall Schematic .....                                                                          | 31 |
| Figure 5: Register file IO .....                                                                           | 35 |
| Figure 6: Register file schematic .....                                                                    | 36 |
| Figure 7: State machine for write operation, same with read operation .....                                | 37 |
| Figure 8: Tmatmul structure .....                                                                          | 40 |
| Figure 9: Pairwise accumulation .....                                                                      | 42 |
| Figure 10: Structure of a 1x512 matrix inside memory .....                                                 | 46 |
| Figure 11: Memory mapping .....                                                                            | 47 |
| Figure 12: Full processor schematic .....                                                                  | 52 |
| Figure 13: Synthesis Result .....                                                                          | 55 |
| Figure 14: Timing result of design .....                                                                   | 56 |
| Figure 15: Operating frequency .....                                                                       | 56 |
| Figure 16: Step by step testcase state machine .....                                                       | 59 |
| Figure 17: Waveform fetch instruction .....                                                                | 59 |
| Figure 18: Waveform TMatMul fetch instruction .....                                                        | 60 |
| Figure 19: Waveform TMatMul parallel processing .....                                                      | 60 |

Figure 20: Comparision between ternary multiplication (left) and normal matrix multiplication (right) .....61

Figure 21: Comparision between ternary multiplication (left) and normal matrix multiplication (right) .....61

**LIST OF TABLES**

Table 1 : Research gap .....7

Table 2: Comparision between Matmul-base and Matmul-free.....22

Table 3: Instruction set .....33

Table 4: Hardware usage .....55

# 1. INTRODUCTION

## 1.1 Motivation

The rapid evolution of large language models (LLMs) has transformed the landscape of artificial intelligence, enabling breakthroughs in tasks such as text generation, translation, summarization, and question answering. Models such as GPT-4, PaLM, and LLaMA have set new benchmarks in performance and versatility. However, these advancements come with significant computational and infrastructural costs. The training and deployment of such models demand massive memory bandwidth, specialized hardware, and enormous energy consumption.

For instance, OpenAI's CEO, Sam Altman [1], indicated at an MIT event in July that the cost of training GPT-4 exceeded \$100 million. With inference latency and power consumption remaining critical bottlenecks for real-time applications. These challenges intensify as model sizes scale beyond 100 billion parameters, necessitating the development of more efficient model architectures and hardware accelerators.

One of the main contributors to computational inefficiency in LLMs is matrix multiplication (MatMul), which dominates both dense layers and attention mechanisms. Despite their effectiveness, MatMul operations impose high silicon area, memory bandwidth, and energy demands, particularly on edge devices or custom hardware platforms such as FPGAs.

This thesis proposes a MatMul-free approach for LLM token generation, supported by an FPGA-based RISC-style coprocessor, to address these inefficiencies and enable scalable, sustainable language modeling.

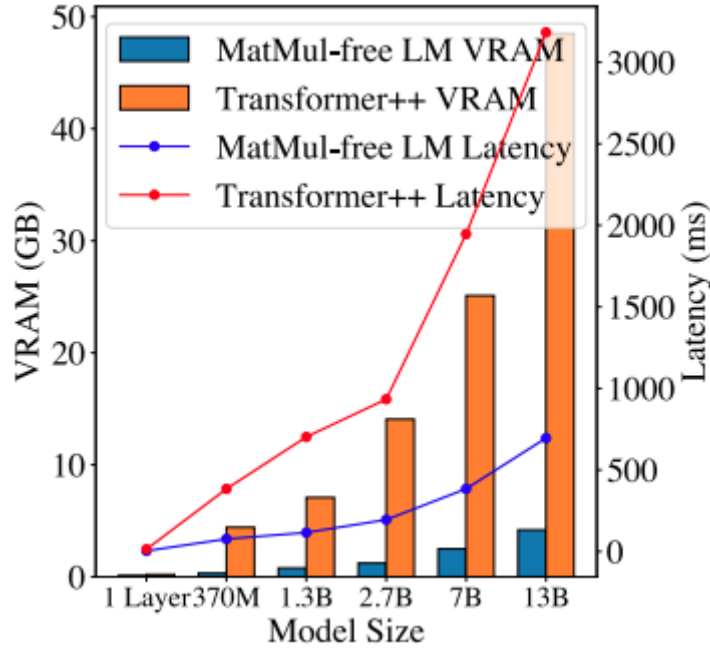


Figure 1: Comparison of GPU Memory and Latency in inference between MatMul-free LM and Transformer++

These figures highlight the escalating expenses associated with developing more sophisticated AI models. As AI models continue to grow in size and complexity, the associated training costs are expected to increase, potentially reaching billions of dollars in the near future. This trend underscores the need for more efficient training methods and hardware to manage the escalating financial and computational demands.

## 1.2 Background

In recent years, the field of natural language processing (NLP) has been revolutionized by the advent of large language models (LLMs). These models, built upon the transformer architecture introduced by Vaswani et al. in 2017 [2], have demonstrated remarkable proficiency in a wide range of tasks such as machine translation, summarization, sentiment analysis, and question answering. Central to the success of LLMs is their reliance on high-dimensional vector operations, particularly dense matrix multiplications (MatMul), which are ubiquitous across all layers, from embeddings and attention to feedforward networks.

Traditionally, LLMs rely heavily on linear transformations such as:

$$Y = XW + b$$

Where  $X$  is the input matrix,  $W$  the weight matrix, and  $b$  the bias vector. These operations are implemented as dense matrix multiplications, often utilizing high-precision floating-point formats (e.g., FP32 or BF16)

However, the very computational operations that enable LLMs to function, specifically MatMul, also constitute a major barrier to their scalability and deployment. In modern transformer-based LLMs, matrix multiplications account for the majority of the computational workload and energy consumption. For instance, in each attention layer, the computation of attention scores through the dot-product of the query and key matrices involves expensive matrix-matrix multiplications (MMM), while feedforward layers typically rely on matrix-vector multiplications (MV). These operations are inherently resource-intensive, especially when implemented at scale with billions of parameters.

To address this challenge, several strategies have emerged to improve the efficiency of neural network computations. One popular technique is quantization, which reduces the precision of weights and activations from 32-bit floating-point to lower-bit representations such as 8-bit integers (INT8), 4-bit formats, or even binary and ternary levels [3]. Quantization significantly reduces memory footprint and computational cost, making it attractive for hardware-constrained environments like mobile devices and embedded systems.

Among these efforts, ternary quantization has gained attention due to its simplicity and hardware-friendliness. In this approach, weight values are restricted to a ternary set  $\{-1, 0, +1\}$ , allowing multiplications to be replaced by additions, subtractions, or no operation at all. This simplification transforms matrix multiplications into ternary accumulations, which are substantially more efficient to implement on digital logic circuits. While this drastically reduces arithmetic complexity, it introduces challenges in maintaining model expressiveness and training stability, issues that are typically addressed through advanced training strategies such as quantization-aware training and normalization layers like RMSNorm.

Parallel to quantization, researchers have explored alternative neural architectures that aim to eliminate MatMul operations entirely. These MatMul-free

architectures utilize components like linearized recurrent units (e.g., MLGRU) and Gated Linear Units (GLUs) that depend on element-wise operations and simplified gating mechanisms rather than matrix multiplications. The key idea is to preserve the modeling power of transformers while avoiding their hardware costs.

At the hardware level, MatMul operations also impose a bottleneck. While GPUs and TPUs are designed with matrix multiplications in mind, leveraging parallelism and dedicated tensor cores, they still suffer from high power consumption and memory bandwidth limitations. Moreover, these platforms are not always suitable for edge deployment due to size, cost, and thermal constraints. In contrast, FPGAs offer a flexible, reconfigurable alternative capable of implementing custom datapaths tailored to specific computational needs. By exploiting this flexibility, it is possible to design efficient hardware accelerators that omit MatMul operations and instead rely on lightweight operations like ternary accumulation.

In this context, the integration of MatMul-free models with FPGA-based architectures presents a promising direction for efficient language model inference. By marrying algorithmic innovation with hardware specialization, it becomes possible to achieve both high throughput and low energy consumption, enabling real-time LLM applications in environments where traditional solutions fall short.

"Scalable MatMul-free Language Modeling" has implemented token generation using an FPGA to reduce the processing load on the GPU. This FPGA-based approach efficiently fetches essential matrix data, including weights, input values, biases, and other relevant parameters, directly from RAM. This offloads significant computational overhead from the GPU, enabling more efficient utilization of resources.

Furthermore, the FPGA design includes dedicated custom instructions tailored for fundamental matrix operations such as addition, subtraction, division, and multiplication. Leveraging these capabilities, it seamlessly handles advanced computational tasks critical for machine learning operations, including the calculation of Gated Recurrent Units (GRUs), the exponential softmax activation function, and the sigmoid activation function.

This thesis builds upon this premise by implementing a MatMul-free token generation pipeline using a custom RISC-style instruction set on an FPGA. This work not only investigates the theoretical foundations of MatMul-free computation but also delivers a practical hardware implementation that can support large-scale LLM inference under constrained power and memory conditions.

### 1.3 Research Gap

Despite the growing interest in model compression and low-precision inference, fully MatMul-free architectures for large language models (LLMs) remain underexplored, especially in the context of hardware-level implementation. While significant progress has been made in reducing the computational complexity of dense neural networks, a critical review reveals three major gaps that this thesis aims to address.

#### 1.4.1 Limited Hardware Deployment of MatMul-Free Models

Many recent works, such as BitNet and Scalable MatMul-Free Language Modeling, have demonstrated that ternary and binary quantization can retain competitive accuracy with reduced compute. However, these models are primarily evaluated in GPU-based environments, which are still optimized for traditional matrix multiplications. Even though some attempts have been made to simulate ternary models in software or use low-bit kernels (e.g., BitBLAS, FlashAttention), real hardware acceleration using FPGAs or ASICs is seldom explored.

Existing implementations often stop at kernel-level optimization or software simulation. There is a lack of end-to-end RTL designs that implement token generation using fully MatMul-free logic with quantized dataflow.

#### 1.4.2 Lack of Modular, Scalable Pipelines for Ternary Logic

Most existing FPGA LLM accelerators focus on optimizing full-precision or fixed-point MatMul operations (e.g., systolic arrays, tile-based GEMM). These architectures are not easily adaptable to ternary arithmetic, which relies heavily on element-wise additions and conditional negations instead of dense matrix multiplication.

Moreover, there is little literature on pipeline-structured, RISC-style architectures tailored specifically for language models using ternary logic. The absence of custom instruction sets and dedicated modules (e.g., TMATMUL blocks, RMS normalization, activation logic) for ternary computation limits the ability to deploy lightweight models on FPGAs or embedded systems.

There is no standardized pipeline architecture designed to handle ternary vector operations, memory-mapped vector I/O, and instruction-level control under a unified datapath.

#### 1.4.3 Insufficient Benchmarking on Real Token Generation Workloads

Most prior benchmarks evaluate compressed models on traditional datasets (e.g., GLUE, SuperGLUE, or causal loss curves). These results, while useful for validating performance at a macro level, do not capture the token-by-token generation dynamics critical in LLM inference.

Furthermore, latency, throughput, and power measurements are often derived from simulation or kernel-level profiling. There is minimal research that:

- + Profiles the actual token throughput per second under hardware constraints
- + Breaks down the latency contribution per hardware stage
- + Analyzes the energy efficiency at inference scale for quantized token generation

A practical system that completes forward-pass token generation in a pipelined, hardware-accelerated fashion has not been fully demonstrated or benchmarked against GPU inference.

#### 1.4.4 Summary of the Research Gap

| Challenge                                    | Current Status                | Limitation              | Research Direction                          |
|----------------------------------------------|-------------------------------|-------------------------|---------------------------------------------|
| <b>Hardware acceleration of ternary LLMs</b> | Mostly simulated or GPU-based | No FPGA implementations | Design MatMul-free token generation on FPGA |

| Challenge                               | Current Status             | Limitation                             | Research Direction                               |
|-----------------------------------------|----------------------------|----------------------------------------|--------------------------------------------------|
| Scalable architecture for ternary logic | Ad-hoc or layer-specific   | No unified RISC-like pipeline          | Develop modular 5-stage pipeline for ternary ops |
| Token generation benchmarking           | Dataset-level metrics only | No inference-level throughput analysis | Benchmark token throughput and latency on FPGA   |

Table 1 : Research gap

### 1.4 Research Objective

The primary objective of this research is to design, implement, and evaluate a MatMul-free coprocessor architecture tailored for token generation in large language models (LLMs). The project targets hardware-level acceleration on FPGA platforms, aiming to improve inference speed, memory efficiency, and energy consumption in resource-constrained environments.

#### 1.4.1. Core Goals

The research is guided by the following core goals:

##### **Eliminate Matrix Multiplication Overhead**

Traditional transformer models rely heavily on high-precision matrix multiplication (MatMul) for dense and attention layers. These operations are computationally intensive and memory-bound. This work seeks to replace all MatMul operations with ternary accumulations, leveraging ternary weight matrices ( $\{-1, 0, +1\}$ ) and addition-only computation pipelines.

##### **Design a Custom 5-Stage RISC-style Pipeline**

The proposed system mimics the classic RISC-V pipeline architecture (Fetch, Decode, Execute, Memory, Write Back), customized to support vectorized instructions and matrix operations. The instruction set includes arithmetic operations (ADD, SUB,

DIV, EXP), memory commands (LDV, STV), and custom TMATMUL (Ternary Matrix Multiplication) instructions.

### **Optimize Memory Bandwidth and Access Patterns**

External DRAM access often becomes a bottleneck in FPGA designs. This project integrates a custom dual-port memory controller and burst-aligned memory mapping scheme to manage concurrent reads/writes efficiently for matrix multiplication. The goal is to support real-time matrix operations with minimal stalling or backpressure.

### **Achieve High Performance on FPGA**

The architecture is synthesized and implemented on a Xilinx Kintex-7 FPGA. Performance metrics including logic utilization, BRAM usage, clock frequency, latency, throughput, and power consumption are carefully measured. A critical objective is to demonstrate significant energy and memory efficiency gains over MatMul-based implementations, while sustaining reasonable throughput (tokens/sec).

#### **1.4.2. Research Questions**

To guide the technical development, this research seeks to answer:

- + Can a fully MatMul-free pipeline, driven by ternary quantization, match or approach the inference efficiency of traditional LLMs on hardware?
- + How effective is a RISC-style instruction set in orchestrating complex language model operations?
- + What are the trade-offs in latency, power, and area when replacing MatMul with ternary accumulation in large-scale vectorized inference?
- + How scalable is the architecture across different model sizes (e.g., 370M, 1.3B parameters) and sequence lengths?

#### **1.4.3. Expected Contributions**

This research is expected to contribute:

- + A hardware-level implementation of MatMul-free token generation tailored for LLMs.
- + A custom instruction set and RISC-style pipeline supporting vectorized and ternary operations.
- + An open-source RTL design that can be used as a reference accelerator for energy-efficient NLP tasks on FPGAs.
- + Empirical benchmarks and comparisons with conventional MatMul-based inference pipelines (GPU and FPGA).

### 1.5 Structure of the Thesis

The remainder of this document is organized as follows:

- + Chapter 2: Related Works reviews existing techniques in quantized and MatMul-free models and their limitations on hardware.
- + Chapter 3: Theory covers the mathematical foundations behind ternary quantization, normalization, and token generation.
- + Chapter 4: Hardware Design details the implementation of the FPGA-based MatMul-free coprocessor.
- + Chapter 5: Evaluation presents performance metrics, synthesis results, and benchmarking.
- + Chapter 6: Conclusion summarizes key findings and proposes future work directions.

## 2. RELATED WORKS

The design of hardware-efficient large language models (LLMs), particularly those eliminating traditional matrix multiplication (MatMul), has gained significant attention due to the growing demand for computationally efficient and scalable architectures. This section reviews prior efforts across three major themes: MatMul-free architectures [4], quantization and low-precision computing [3], and FPGA-based language model acceleration.

### 2.1 Scalable MatMul-free Language Modeling

Matrix multiplication (MatMul) has long been a central operation in deep learning, serving as the foundation of dense layers, attention mechanisms, and convolutional operations. However, MatMul is also one of the most resource-intensive components in neural networks, especially in large-scale language models (LLMs) where embedding sizes and layer depths are substantial. The cost of MatMul manifests in high memory bandwidth requirements, significant energy consumption, and hardware inefficiencies—particularly for edge deployment or FPGA-based implementation. As a result, recent research has explored alternative architectures that eliminate or reduce reliance on MatMul while retaining strong performance. These approaches are collectively referred to as *MatMul-free architectures*.

One of the most influential contributions to this field is the **Scalable MatMul-free Language Modeling** framework by Zhu et al [4]. This work introduces the first LLM architecture that eliminates MatMul operations entirely while maintaining competitive accuracy at the billion-parameter scale. Their architecture achieves this by systematically replacing all multiplications with ternary accumulations, element-wise operations, and simple arithmetic logic, implemented across both token and channel mixers.

#### **Ternary Dense Layers (BitLinear Layers)**

The cornerstone of MatMul-free architectures is the **BitLinear layer**, which replaces the standard matrix multiplication in dense layers with ternary accumulation. In this formulation, the weight matrix is quantized to contain only three values:  $\{-1, 0,$

$+1\}$ . This dramatically reduces computational complexity by eliminating multiplications altogether, since any input value can be added, subtracted, or ignored. This approach leads to substantial savings in energy, memory bandwidth, and area on digital hardware.

Let  $W \in \{-1,0,+1\}^{d \times m}$  be the quantized ternary weight matrix, and let  $x \in \mathbb{R}^{1 \times d}$  be the input vector. The output  $y \in \mathbb{R}^{1 \times m}$  is computed via ternary accumulation:

$$y_i = \sum_{j=1}^d x_j W_{ij}, \quad \text{where } W_{ij} \in \{-1,0,+1\}$$

This operation simplifies to additions and subtractions based on the sign of  $W_{ij}$ , which can be executed efficiently on hardware.

### **Token Mixing Without Attention (MLGRU)**

Traditional self-attention mechanisms rely heavily on matrix multiplications between query (Q), key (K), and value (V) matrices. These computations dominate inference time and power consumption in transformers. To eliminate this cost, Zhu et al. propose the MatMul-free Linear Gated Recurrent Unit (MLGRU), a recurrence-based token mixer that uses only element-wise operations and ternary accumulation.

In the MLGRU formulation, all hidden-to-hidden transitions and MatMul operations are removed. The remaining computations involve linear transformations of input vectors using ternary weights, followed by gating with sigmoid or SiLU (Sigmoid Linear Unit) activations. The absence of MatMul allows the unit to be highly parallelizable and efficient on custom hardware:

$$f_t = \sigma(x_t \odot W_f + b_f), \quad c_t = \tau(x_t \odot W_c + b_c)$$

$$h_t = f_t \odot h_{t-1} + (1 - f_t) \odot c_t$$

$$o_t = (\sigma(x_t \odot W_g + b_g) \odot h_t) \odot W_o + b_o$$

Here,  $\odot$  denotes ternary accumulation, and  $\tau$  is a lightweight activation (e.g., SiLU). The model retains the ability to capture temporal dependencies while operating under strict hardware constraints.

### GLU-Based Channel Mixing with Ternary Weights

In addition to temporal dependencies, LLMs must model cross-dimensional interactions, typically handled by feedforward networks. MatMul-free architectures achieve this via **Gated Linear Units (GLUs)** implemented using BitLinear layers. These GLUs replace each dense projection with ternary matrix multiplication, up-projecting the input dimension, applying gating via activations, and projecting back down—all without using MatMul:

$$g_t = x_t \circledast W_g$$

$$u_t = x_t \circledast W_u$$

$$p_t = \tau(g_t) \odot u_t$$

$$d_t = p_t \circledast W_d$$

This operation structure enables both gating and transformation while maintaining computational simplicity. Notably, BitLinear GLUs preserve much of the expressiveness of standard dense layers due to their gating nature, despite using only ternary weights.

### Comparison to Other Approaches

While previous attempts such as AdderNet have explored multiplier-free operations for vision models using signed additions, their designs were not targeted toward sequential or linguistic data, and were not optimized for quantized transformer layers.

More recent works like BitNet and RWKV-4 employ binary or ternary quantization to reduce weight precision, but often retain attention mechanisms involving matrix multiplications. For example, BitNet replaces dense layers with ternary ones, but attention matrices are still MatMul-based, creating a bottleneck. In contrast, the MLGRU-based token mixer proposed in MatMul-free architectures avoids all multiplications, offering a truly MatMul-free pipeline suitable for hardware deployment.

## 2.2 Quantization and Low-Precision Language Models

The growing size and complexity of large language models (LLMs) have spurred interest in quantization techniques that reduce computational overhead and memory footprint. Quantization converts high-precision weights and activations (typically float32 or bfloat16) into lower-precision representations such as 8-bit integers, ternary values  $\{-1, 0, +1\}$ , or even binary values  $\{-1, +1\}$ . These techniques not only reduce model size but also enable efficient execution on custom hardware such as FPGAs and ASICs.

### 2.2.1 Full-Precision vs. Quantized Models

Traditional LLMs like GPT-3 and LLaMA utilize high-precision operations that consume significant power and memory, especially during inference on edge or embedded devices. As a result, quantization has emerged as a key enabler for deploying these models in resource-constrained environments. Quantization reduces arithmetic complexity and allows for faster computation through bitwise operations and smaller data movement.

For instance, an unquantized matrix multiplication involves thousands of float32 multiply-accumulate (MAC) operations, which are not only energy-hungry but also require wide memory buses. In contrast, a quantized multiplication using ternary weights avoids multipliers entirely, reducing the operation to simple additions, subtractions, or skips.

### 2.2.2 Ternary Quantization

Ternary Quantization is a compromise between binary quantization and full-precision weights. It restricts weights to one of three values:  $-1$ ,  $0$ , or  $+1$ . This ternary constraint eliminates multipliers and significantly reduces memory access cost.

The ternary weight matrix  $\tilde{W}_{ij} \in \{-1, 0, +1\}$  replaces the full-precision matrix  $W \in \mathbb{R}^{d \times m}$ , and the multiplication is transformed into a scaled accumulation:

$$y_i = \sum_{j: \tilde{W}_{ij}=1} x_j - \sum_{j: \tilde{W}_{ij}=-1} x_j$$

This operation requires no floating-point multipliers and can be efficiently mapped to parallel add-subtract trees in hardware.

To obtain ternary weights, a quantization scheme such as abs-mean quantization is used during training:

$$\tilde{W}_{ij} = RoundClip\left(\frac{W_{ij}}{\gamma + \epsilon}, -1, 1\right), \gamma = \frac{1}{nm} \sum_{ij} |W_{ij}|$$

This scaling ensures that the quantized weights maintain the same statistical distribution as the original.

### 2.2.3 Activation Quantization and RMS Normalization

In addition to weights, activations are also quantized to 8-bit or lower precision using techniques such as absmax scaling:

$$\tilde{x} = Quant(x) = Clip\left(\frac{x \cdot Q_b}{\|x\|_\infty}, -Q_b, +Q_b\right), \quad Q_b = 2^{b-1}$$

To stabilize the training and prevent gradient vanishing or explosion, RMS Normalization (RMSNorm) is applied prior to quantization. This normalization ensures that input vectors have unit variance:

$$RMSNorm(x) = \frac{x}{\sqrt{\mathbb{E}[x^2] + \epsilon}}$$

These normalization and quantization techniques, when fused and implemented on-chip as in our design, reduce the latency and overhead typically caused by off-chip memory accesses.

### 2.2.4 Ternary LLMs in Literature

Several recent works have demonstrated the feasibility of ternary and binary quantization in transformer-based language models:

- + BitNet (Wang et al., 2023): Proposed a family of ternary transformers that scale up to 3 billion parameters. Despite using only ternary weights in dense layers, BitNet preserves performance close to its full-precision counterparts.

- + BinaryBERT (Bai et al., 2020): Demonstrated the possibility of binarizing BERT models through knowledge distillation and post-training quantization.
- + BiBERT and BitDistiller: Extended these techniques by progressively quantizing transformer layers and using student-teacher frameworks to preserve accuracy.

However, most of these works retain MatMul in attention blocks and rely on GPU-optimized kernels, which still pose challenges for deployment on non-GPU platforms. Our work diverges by fully eliminating MatMul and implementing quantized arithmetic natively in hardware.

### 2.3 FPGA Acceleration for LLMs

As the computational demands of large language models (LLMs) continue to grow, field-programmable gate arrays (FPGAs) have emerged as a viable alternative to GPUs and ASICs for efficient and customizable inference. Their ability to perform fine-grained, parallel computation with low latency and power consumption makes them particularly attractive for edge deployment and specialized neural architectures, such as MatMul-free language models.

Traditional LLM accelerators, including Google's Tensor Processing Units (TPUs), NVIDIA's Transformer Engine, and Xilinx's Deep Processing Unit (DPU), are primarily optimized for high-throughput matrix multiplication. These platforms leverage large, fused multiply-add arrays and are designed to support high-precision operations such as BF16 or FP32. While effective, they tend to exhibit inefficiencies when executing non-standard or sparsely-connected architectures due to rigid hardware pipelines and fixed instruction sets.

In contrast, FPGAs offer architectural flexibility that enables the implementation of non-traditional models such as binary or ternary neural networks. Recent work by Zhu et al. (2024) introduced a pioneering MatMul-free FPGA accelerator that replaces dense matrix multiplications with ternary accumulations and element-wise operations. Their architecture features four functional units, for row-wise operations, root mean square normalization, ternary matrix multiplication, and load/store, orchestrated through a custom instruction set and pipelined RISC control logic. Deployed on Intel's

Stratix 10 FPGA, this design achieved 43 ms inference latency for a single token generation block with only 13W average power, demonstrating promising performance and energy efficiency.

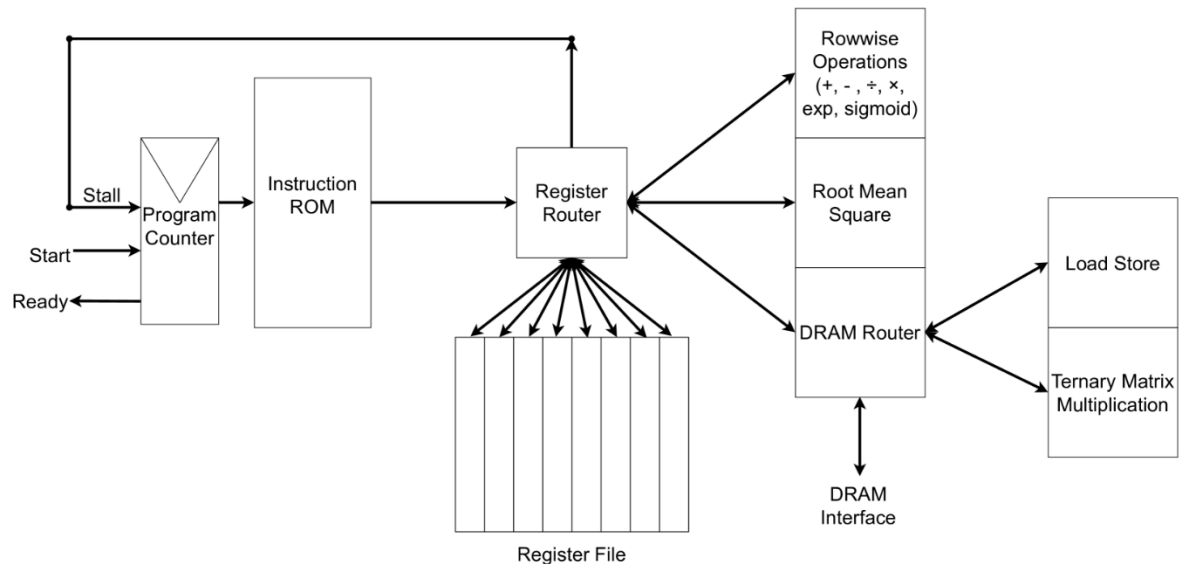


Figure 2: RTL implementation for running MatMul-free token generation

### 3. THEORY

#### 3.1 Token generation

Tokenization is the process of converting raw text into smaller units called tokens. These tokens can be words, subwords, or even individual characters, depending on the tokenizer's design. For instance, the sentence "The cat sat on the mat." might be tokenized into ["The", "Ġcat", "Ġsat", "Ġon", "Ġthe", "Ġmat", "."]<sup>1</sup> (example from BPE or SentencePiece).

Each token is then assigned a unique integer ID based on the tokenizer's vocabulary:

```
"The"    → 132
"Ġcat"   → 4567
"Ġsat"   → 2231
"Ġon"    → 1021
"Ġthe"   → 132
"Ġmat"   → 5844
"."      → 4
```

So the sentence becomes: [132, 4567, 2231, 1021, 132, 5844, 4]

Once text is tokenized, each token is converted into a vector representation known as an embedding. Embeddings are dense numerical vectors that capture the semantic and syntactic properties of tokens. In these high-dimensional spaces, tokens with similar meanings are positioned closer together, allowing the model to understand relationships between words.

These embeddings are generated through training on large corpora, enabling the model to learn contextual nuances and associations between words. For example, the words "The" and "cat" would have embeddings that reflect their related meanings, differing in specific dimensions that capture gender or role distinctions.

These token IDs are then mapped into **embedding vectors** using an **embedding matrix**, which is a learnable parameter of the model.

For each token ID, the corresponding row in the embedding matrix is fetched:

```
132    → [0.12, -0.45, ..., 0.08]    # 768-dim vector
```

---

<sup>1</sup> The "Ġ" prefix denotes a space before the token; it's common in tokenizers like RoBERTa.

```
4567 → [0.21, 0.14, ..., -0.39]
        ...
```

So the sequence of token IDs transforms into a **matrix** of shape (**7 tokens × 768 dimensions**) — this is the **input embedding** matrix.

Now the model processes this matrix through its multiple layers (attention layers, feedforward layers, etc.). Each layer operates on these embeddings, updating them to capture deeper syntactic and semantic relationships.

Let's say this is a **language generation** task (like autocomplete). After processing, the model outputs logits over the vocabulary for the next token prediction.

```
Output logits → softmax → predicted next token: "Ġand" (ID: 945)
```

If the model is generating a response, it will repeat this process for the next token using its own output as input — eventually decoding the generated token IDs back into readable text like:

```
["Ġand", "Ġran", "Ġaway", "."]
```

Which decodes to: "**and ran away.**"

This pipeline allows LLMs to perform a variety of language tasks, such as translation, summarization, and question answering, by effectively understanding and generating human language.

### 3.2 The Cost of Matrix Multiplication

Matrix multiplication (MatMul) forms the computational backbone of nearly all large-scale neural network architectures, including transformers used in large language models (LLMs). Despite its widespread adoption, MatMul is also the single most resource-intensive operation, posing serious challenges in terms of energy consumption, hardware efficiency, and latency, particularly for edge deployment or real-time inference.

#### 3.2.1 Mathematical Formulation

The fundamental form of matrix multiplication in neural networks is expressed as:

$$\mathbf{Y} = \mathbf{X} \cdot \mathbf{W}$$

Where:

- +  $\mathbf{X} \in R^{1 \times d}$  is the input row vector (e.g., token embedding).
- +  $\mathbf{W} \in R^{d \times m}$  is the learned weight matrix of the layer.
- +  $\mathbf{Y} \in R^{1 \times m}$  is the output vector of the transformation.

The number of floating-point operations (FLOPs) required for such a computation is  $2dm$ , where each output element involves  $d$  multiplications and  $d-1$  additions. For a typical transformer layer with  $d=2048$  and  $m=2048$ , each forward pass over one token involves approximately 8.4 million operations.

When stacked across dozens of layers and repeated for each token in a sequence, the computational load becomes substantial, often exceeding teraflops per second (TFLOPS) during inference. This makes MatMul the primary bottleneck in both compute and memory subsystems.

### 3.2.2 Hardware Implications

From a hardware design perspective, matrix multiplication demands:

- + Multiply-Accumulate Units (MACs): Each consisting of a multiplier followed by an adder, these are expensive in silicon area and power.
- + High-precision data paths: Floating-point (FP16, FP32) or fixed-point multipliers dominate logic utilization on FPGAs and ASICs.
- + Large memory bandwidth: All operands must be fetched from memory, which increases latency and energy per operation.

On FPGA platforms, multipliers consume DSP slices or ALMs (Adaptive Logic Modules), which are limited in number. Even with parallelization, maintaining high throughput across many MACs often requires:

- + Clock frequency throttling (to reduce thermal load),
- + Complex scheduling of memory access,
- + And pipeline depth optimization to hide latency.

Moreover, GPUs and CPUs optimized for MatMul (e.g., via cuBLAS, oneDNN) depend heavily on cache locality, and their performance drops significantly when memory access becomes non-contiguous—common in high-dimensional LLMs.

### 3.2.3 Energy and Latency Overhead

In modern hardware:

- + A 16-bit floating-point multiplication can consume  $3\text{--}5\times$  more energy than a simple addition or bitwise operation.
- + Each MatMul operation incurs dynamic energy costs for both computation and DRAM access.

Experimental studies show that over 90% of energy used during inference of LLMs goes to MatMul operations in attention and feedforward layers. Latency per token can range from 10 ms to 40 ms, depending on model size and sequence length, with MatMul dominating the critical path.

This becomes particularly problematic for edge-AI deployments, where energy budgets are limited, and real-time latency is essential. The need to process billions of parameters under such constraints necessitates alternative computation strategies.

### 3.2.4 Scalability Challenges

As language models scale up from hundreds of millions to hundreds of billions of parameters (e.g., GPT-3: 175B), the proportion of resources spent on MatMul grows superlinearly:

- + Quadratic growth in attention MatMuls with respect to sequence length  $L$ .
- + Linear scaling in dense layer MatMuls with respect to model width  $d$  and depth  $N$ .

This scaling trend exacerbates hardware inefficiencies, and even with aggressive quantization (e.g., INT8, FP8), multiplication-heavy layers remain dominant in latency profiles.

To overcome these limitations, the proposed architecture completely replaces all MatMul operations with ternary accumulation. In this approach, the weight matrix  $W$

is constrained to ternary values from the set  $\{-1,0,+1\}$ , and the multiplication is replaced by an equivalent set of additions and subtractions:

$$y_i = \sum_{j:\tilde{W}_{ij}=1} x_j - \sum_{j:\tilde{W}_{ij}=-1} x_j$$

Where:

- +  $\tilde{W}_{ij}$  denotes the ternary weight,
- + The operation requires no multipliers, only additions or negations.
- + This reformulation introduces a number of compelling benefits:
- + Multipliers are eliminated, replaced with simple adders and inverters.
- + Hardware complexity is greatly reduced, allowing the design to fit within low-cost FPGAs or lightweight ASICs.
- + Energy efficiency is dramatically improved, as additions consume less than 10% of the energy of equivalent multiplications.
- + Quantized weights are memory-efficient, requiring only 2 bits per element, enabling higher model capacity within the same memory budget.

### 2.3.3 Hardware Implications

The removal of multipliers enables a fully custom data path, where each processing unit can operate at higher clock frequencies or lower voltages. This is particularly advantageous for FPGAs, where logic resources are often limited and power budgets are tight.

With no floating-point units or large memory fetches required, the system supports deeper pipelining and finer-grained parallelism, further enhancing throughput and scalability. This enables the MatMul-free coprocessor to be deployed in edge AI scenarios, IoT devices, or battery-powered inference platforms where conventional LLMs are infeasible.

| Attribute                   | MatMul-based LLM          | MatMul-Free Architecture |
|-----------------------------|---------------------------|--------------------------|
| Operation Type              | Multiply-Accumulate       | Add/Sub Only             |
| Weight Precision            | 16- or 32-bit             | Ternary (2-bit)          |
| Hardware Requirements       | FP multipliers, large RAM | Adders, small SRAM       |
| Energy Efficiency           | Low                       | High                     |
| Memory Bandwidth            | High                      | Low                      |
| Edge Deployment Feasibility | Difficult                 | Highly Feasible          |

Table 2: Comparison between Matmul-base and Matmul-free

### 3.3 BitLinear Layers and Ternary Quantization

BitLinear layers are the foundational building blocks of MatMul-free architectures. Unlike conventional dense layers that rely on full-precision matrix multiplications, BitLinear layers operate entirely with ternary weights, where each weight is constrained to one of three values:  $-1$ ,  $0$ , or  $+1$ . This drastically reduces computational complexity, hardware cost, and memory bandwidth, making the architecture well-suited for deployment in energy-constrained and resource-limited environments such as FPGAs.

#### 3.3.1 Ternary Weight Quantization

The transformation from a full-precision weight matrix  $\mathbf{W}$  to a ternary matrix  $\widetilde{\mathbf{W}}$  is achieved through a process called abs-mean quantization. The goal is to preserve

the relative significance of weights while mapping them to a discrete ternary set. The quantization is defined as:

$$\tilde{W}_{ij} = \begin{cases} +1 & \text{if } W_{ij} > \Delta \\ 0 & \text{if } |W_{ij}| \leq \Delta \\ -1 & \text{if } W_{ij} < -\Delta \end{cases} \quad \text{where } \Delta = \text{mean}(|W|)$$

This method ensures that only the most influential weights are retained as non-zero values, which improves efficiency while maintaining expressivity. The threshold  $\Delta$  can be computed per-layer or per-block to balance sparsity and accuracy.

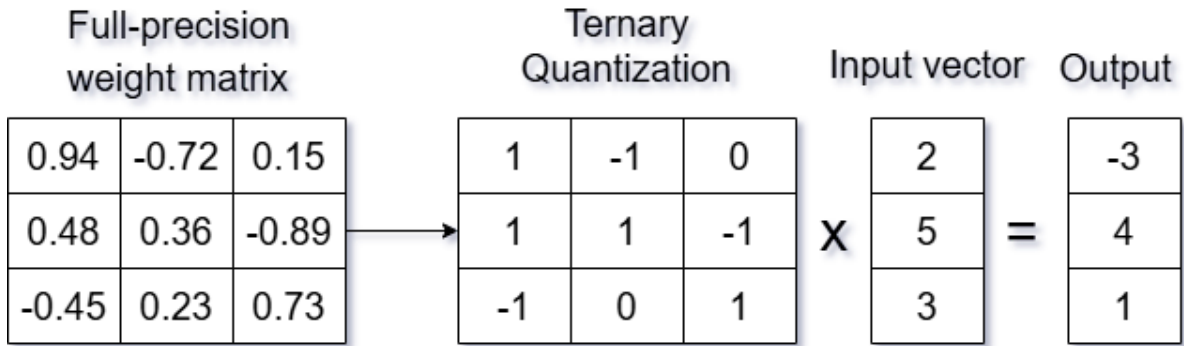


Figure 3: Visual illustration of ternary quantization and BitLinear layers

A full-precision weight matrix (left) is transformed into a ternary weight matrix (center) by mapping each element to  $\{-1, 0, +1\}$ . This ternary matrix is then multiplied by an input vector using addition and subtraction only, producing the output vector (right). This transformation eliminates the need for expensive multipliers, enabling efficient implementation on FPGA hardware. The quantized matrix enables lightweight addition/subtraction operations instead of costly multiplications, forming the basis of BitLinear layers. During inference, each row of the ternary weight matrix is element-wise multiplied with the input vector, accumulating the results efficiently using simple hardware adders. This forms the foundation for efficient MatMul-free architecture deployment in large language models.

### 3.3.2 Quantized Activation Processing

In addition to quantizing weights, activations are also reduced to lower precision to optimize performance and memory usage. A common technique is abs-max

quantization to 8-bit integers, where the scaling factor is derived from the maximum absolute value of the activation vector:

$$s = \frac{127}{\max(|\mathbf{x}|)}, \quad \tilde{x}_j = \text{Clip}(\text{Round}(s \cdot x_j), -128, 127)$$

This normalization and quantization step is typically applied after RMSNorm, which stabilizes the activation distribution. The normalized input  $\mathbf{x}'$  is computed as:

$$\mathbf{x}' = \frac{\mathbf{x}}{\sqrt{\mathbb{E}[\mathbf{x}^2] + \epsilon}}$$

This process ensures that activations are within a suitable dynamic range for quantization, which in turn maintains numerical stability and gradient flow during training.

### 3.3.3 Fused BitLinear Layer

To further optimize inference and training efficiency, the quantization, normalization, and accumulation steps are fused into a single operation. This eliminates redundant memory accesses and enables better data locality, especially in SRAM-based buffers in custom hardware implementations. The forward computation of a fused BitLinear layer can be summarized as:

$$\tilde{\mathbf{y}} = \text{Quant}(\text{RMSNorm}(\mathbf{x})) \circledast \tilde{\mathbf{W}} + \mathbf{b}$$

where  $\circledast$  denotes ternary accumulation. This fused pipeline reduces latency and improves throughput, especially when deployed in pipelined or streaming architectures.

## 3.4 MLGRU: A MatMul-Free Token Mixer

Modern large language models (LLMs) rely heavily on self-attention mechanisms to capture temporal dependencies between tokens. However, self-attention requires multiple matrix multiplications (MatMuls) across the query (Q), key (K), and value (V) matrices operations that are costly in terms of computation, memory bandwidth, and power consumption, especially on custom hardware like FPGAs. In a MatMul-free architecture, an alternative method is required to fulfill the role of token

mixing without relying on multiplications. This section presents the MatMul-free Linear Gated Recurrent Unit (MLGRU), a novel token mixing module that eliminates all MatMul operations by relying solely on element-wise operations and ternary accumulations.

### 3.4.1 Motivation

In transformer-based architectures, token mixers are typically implemented via self-attention blocks, which compute:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

This process includes at least two dense matrix multiplications and a softmax operation, inefficient on low-power hardware. Moreover, the dependency of the attention output on dynamically computed Q and K matrices makes it challenging to apply quantization without accuracy degradation.

To address these limitations, we adopt a simplified recurrent structure based on the Gated Recurrent Unit (GRU), re-engineered for fully MatMul-free operation while preserving the ability to learn temporal dependencies across sequences.

### 3.4.2 Standard GRU Overview

The standard GRU involves multiple gating mechanisms and hidden-to-hidden transformations, defined as:

$$r_t = \sigma(x_t W_{xr} + h_{t-1} W_{hr} + b_r)$$

$$f_t = \sigma(x_t W_{xf} + h_{t-1} W_{hf} + b_f)$$

$$\tilde{h}_t = \tanh(x_t W_{xc} + (r_t \odot h_{t-1}) W_{hc} + b_c)$$

$$h_t = f_t \odot h_{t-1} + (1 - f_t) \odot \tilde{h}_t$$

Each of these equations includes multiple matrix multiplications, which contradicts the objective of a MatMul-free pipeline.

### 3.4.3 MLGRU Formulation

In the proposed MLGRU, all matrix multiplications are replaced by ternary accumulations, and hidden-to-hidden transformations are removed to enable parallel computation. The final architecture is defined as:

$$f_t = \sigma(x_t \tilde{\otimes} W_f + b_f)$$

$$c_t = \tau(x_t \tilde{\otimes} W_c + b_c)$$

$$h_t = f_t \odot h_{t-1} + (1 - f_t) \odot c_t$$

$$g_t = \sigma(x_t \tilde{\otimes} W_g + b_g)$$

$$\hat{o}_t = g_t \odot h_t$$

$$o_t = \hat{o}_t \tilde{\otimes} W_o + b_o$$

Where:

- +  $\tilde{\otimes}$  denotes ternary accumulation: addition/subtraction only.
- +  $\odot$  is element-wise multiplication.
- +  $\sigma(\cdot)$  is the sigmoid activation.
- +  $\tau(\cdot)$  is the SiLU (Sigmoid-Weighted Linear Unit) activation.
- + All weights  $W$  are ternary:  $\{-1, 0, +1\}$ .

### 3.4.4 Computational Simplification

Each ternary accumulation replaces multiplication with:

$$x_j \cdot w_{ij} = \begin{cases} +x_j, & \text{if } w_{ij} = +1 \\ 0, & \text{if } w_{ij} = 0 \\ -x_j, & \text{if } w_{ij} = -1 \end{cases}$$

This transformation enables the model to use only adders and subtractors for the entire token mixer, drastically reducing hardware cost and latency.

### 3.5 MatMul-Free Channel Mixer: BitLinear-GLU

In large language models (LLMs), channel mixers are used to capture interactions **across embedding dimensions** within each token. In transformer architectures, this role is typically fulfilled by feedforward networks (FFNs), which consist of two dense layers with an activation function in between. However, the standard implementation relies heavily on matrix multiplication (MatMul), which poses a challenge for deployment in energy-constrained or memory-limited environments.

To address this, our design replaces conventional FFNs with a **MatMul-free Gated Linear Unit (GLU)** using ternary weights, a structure we refer to as **BitLinear-GLU**. This module enables efficient cross-dimensional mixing without requiring multipliers, making it ideal for hardware acceleration on FPGAs and custom ASICs.

#### 2.6.1 Overview of Gated Linear Units (GLUs)

The Gated Linear Unit is a widely adopted channel mixing mechanism in modern LLMs such as LLaMA, Mistral, and RWKV. It provides a balance between nonlinear expressiveness and computational efficiency through a gating mechanism. A typical GLU operates as follows:

$$\text{GLU}(x) = (xW_u) \odot \tau(xW_g)$$

Where:

- +  $x \in R^{1 \times d}$  is the input token embedding,
- +  $W_u, W_g \in R^{d \times l}$  are the up-projection weight matrices,
- +  $\tau(\cdot)$  is a nonlinear activation function (commonly SiLU or Swish),
- +  $\odot$  denotes element-wise multiplication.

The output is then projected back to the original embedding dimension using another matrix  $W_d \in R^{l \times d}$ , forming the complete channel mixing block:

$$\text{Output} = \left[ (xW_u) \odot \tau(xW_g) \right] W_d$$

This sequence of three MatMuls, up-projection, gating, and down-projection, accounts for a significant portion of computation in transformer models.

### 2.6.2 BitLinear-GLU: MatMul-Free Reformulation

To eliminate MatMul operations, we reformulate GLU using **ternary weight matrices** and replace all dense layers with **BitLinear modules**, which compute ternary accumulations:

Let:

- +  $W_g, W_u, W_d$  be ternary matrices with elements constrained to  $\{-1, 0, +1\}$ ,
- +  $\tilde{\otimes}$  denote ternary accumulation,
- +  $\tau(\cdot)$  be the SiLU activation function.

The BitLinear-GLU is defined as:

$$g_t = x \tilde{\otimes} W_g$$

$$u_t = x \tilde{\otimes} W_u$$

$$p_t = \tau(g_t) \odot u_t$$

$$d_t = p_t \tilde{\otimes} W_d$$

Where:

- +  $g_t$  is the gate vector, activated by SiLU,
- +  $u_t$  is the content vector,
- +  $p_t$  is the gated content,
- +  $d_t$  is the final channel-mixed output.

This formulation removes all floating-point multiplications and reduces each projection step to simple additions and subtractions based on ternary weight values.

### 2.6.4 Parameter and Compute Cost Analysis

Assume an input dimension  $d = 512$  and expansion factor  $r = 8$ . The BitLinear-GLU uses three ternary projections:

$$\begin{aligned}
 &+ W_u \in R^{512 \times 4096} \\
 &+ W_g \in R^{512 \times 4096} \\
 &+ W_d \in R^{4096 \times 512}
 \end{aligned}$$

Total weight storage (ternary):

$$\text{Bits} = (512 \times 4096 \times 2 \times 2 + 4096 \times 512 \times 2) = 16,777,216 \text{ bits} = 2\text{MB}$$

In contrast, FP16 would require  $\sim 12.6$  MB, a  $6.3\times$  increase. Moreover, since all computations use only addition/subtraction, the logic complexity is vastly reduced.

## 4. HARDWARE DESIGN AND IMPLEMENTATION

### 4.1 Overview of the MatMul-Free Coprocessor

RISC architectures, known for their streamlined instruction sets, are well-suited to handle these operations efficiently. The simplicity of RISC designs allows for the integration of custom instructions, enabling faster and more energy-efficient processing. For instance, the xTern system extends the RISC-V instruction set to accelerate ternary neural network inference, achieving significant improvements in 67% higher throughput and 57% higher energy efficiency. [3]

Implementing a MatMul-free architecture on FPGA requires careful optimization to maximize performance while minimizing resource utilization.

#### **Techniques:**

- + **Pipelining:** Pipeline the FPGA implementation to increase the speed of the design, for easier to implement parallel processing, running other instructions while calculating matrix multiplication
- + **Custom Instruction Sets:** Designing FPGA-specific instructions for operations like ternary matrix multiplication and normalization.
- + **Expected Impact:** Enhanced throughput, reduced latency, and improved scalability for language modeling tasks.

Our design structured similarly to a classic 5-stage pipeline (Fetch → Decode → Execute → Memory → Write Back), with some custom modules to support ternary operations and specialized matrix handling. The architecture employs a modular design where each pipeline stage is distinctly separated by pipeline registers, illustrated as vertical bars. This separation facilitates pipelined execution, significantly enhancing system throughput and overall computational efficiency.

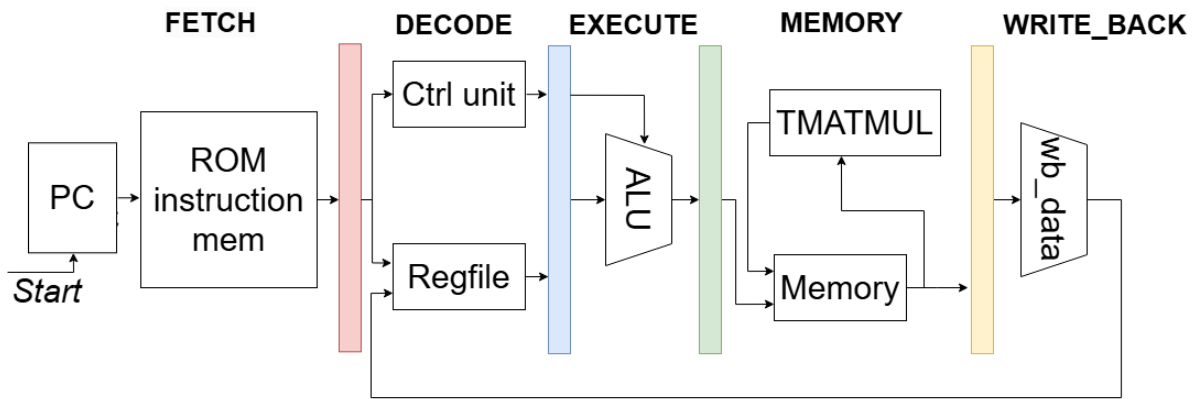


Figure 4: Overall Schematic

- + **FETCH:** This stage involves fetching instructions by holding the address of the current instruction in the Program Counter (PC), structured similarly to an accumulator. ROM Instruction Memory is utilized to store the program containing custom instructions. The PC retrieves instructions directly from the ROM, preparing them for subsequent processing stages.
- + **DECODE:** The Control Unit (Ctrl Unit) decodes the fetched instructions and generates control signals (ctrl\_signal) to identify and activate the appropriate functional units, such as the Arithmetic Logic Unit (ALU), Ternary Matrix Multiplication Unit (TMATMUL), or Memory unit. Concurrently, the Register File (Regfile) contains general-purpose vector registers that provide register data (Reg\_data) to the execution units based on the decoded instructions, ensuring efficient and accurate operation.
- + **EXECUTE:** This stage includes the Arithmetic Logic Unit (ALU), which handles element-wise vector operations such as addition (ADD), subtraction (SUB), multiplication (MUL), division (DIV), activation functions (SIG, EXP), and vector normalization. The ALU operation is guided by the alu\_op control signal and processes input data supplied from the register file.
- + **MEMORY:** TMATMUL (Ternary Matrix Multiplication) block is specialized hardware module is dedicated to performing ternary matrix-vector multiplications. It executes matrix multiplications utilizing ternary accumulations by retrieving the ternary matrix from memory along with the user input vector, subsequently outputting the computed results back to memory for

storage. Memory Unit is responsible for general-purpose memory operations, this unit handles loading and storing vectors from DDR3. It manages the retrieval and provision of ternary or user-defined vectors required for calculations performed within the TMATMUL module.

- + **WRITE\_BACK**: This stage incorporates a multiplexer designed to select the appropriate data source for write-back operations. It chooses between the output generated by the Arithmetic Logic Unit (ALU) or the data retrieved from memory, ensuring the selected output is accurately written back into the register file.

#### 4.2 Risc-based custom instruction set

Base on instruction set of Rui-Jie Zhu's RTL implementation for running MatMul-free token generation, we have custom some instructions exclusive for our design. Each 13-bit instruction operates over vectors, typically of shape  $1 \times 512$  or  $512 \times 512$ , using a row-wise operation model. The processor supports basic arithmetic, normalization, activation functions, and ternary matrix multiplications, tailored to the needs of MatMul-free LLMs. All instructions act on full vectors, suitable for parallel dataflow with fixed-vector size (e.g., 512 elements). The instruction format is 4 bit opcode, and the rest is destination for register or memory.

| Inst | Opcode | Operation      | Description                          |
|------|--------|----------------|--------------------------------------|
| LDV  | 1001   | LDV R1, V1     | Load vector from DRAM to a register  |
| STV  | 1010   | STV V1, R1     | Store vector from a register to DRAM |
| ADD  | 0001   | ADD R3, R2, R1 | Row-wise Addition                    |
| SUB  | 0010   | SUB R3, R2, R1 | Row-wise Subtraction                 |
| MUL  | 0011   | MUL R3, R2, R1 | Row-wise Multiplication              |
| DIV  | 0100   | DIV R3, R2, R1 | Row-wise Division                    |
| EXP  | 0101   | EXP R3, R2     | Row-wise Exponential Function        |

| Inst    | Opcode | Operation          | Description                              |
|---------|--------|--------------------|------------------------------------------|
| SIG     | 0110   | SIG R3, R2         | Row-wise Sigmoid Function                |
| NORM    | 0111   | NORM R3, R2        | Performs RMS normalization on the vector |
| TMATMUL | 1000   | TMATMUL V3, V2, V1 | Ternary Matrix Multiplication            |
| HALT    | 1111   | -                  | Terminate Processor                      |

Table 3: Instruction set

To efficiently support the MatMul-free LLM token generation process, we design a set of specialized functional units that directly map to the operations required by the MatMul-free architecture. These units are integrated into a RISC-style datapath, each optimized for low-latency, low-power execution on an FPGA platform.

## Format instruction for ADD, SUB, MUL, DIV, MATMUL

Instruction: 0001 000 001 010

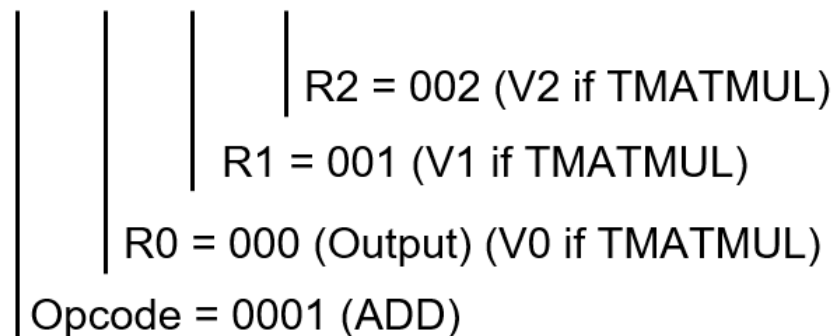


Figure 5: Format for Arithmetic and TMATMUL Instructions

**Figures 5** present the format used for arithmetic operations (**ADD, SUB, MUL, DIV**) and **TMATMUL**. Each instruction is structured with:

- + A 4-bit opcode in the most significant bits, defining the operation type (e.g., ADD = 0001).

- + Three 3-bit register identifiers (R0, R1, R2) specifying the destination and source operands.
  - For general arithmetic instructions:
    - **R0** denotes the destination register for the operation result.
    - **R1** and **R2** denote the source operand registers.
  - For **TMATMUL**:
    - **V0**, **V1**, and **V2** correspond to the ternary matrix output and operand vector registers.

This consistent structure simplifies instruction decoding and supports both vector arithmetic and matrix operations within a unified pipeline architecture.

## Format instruction for LDV, STV

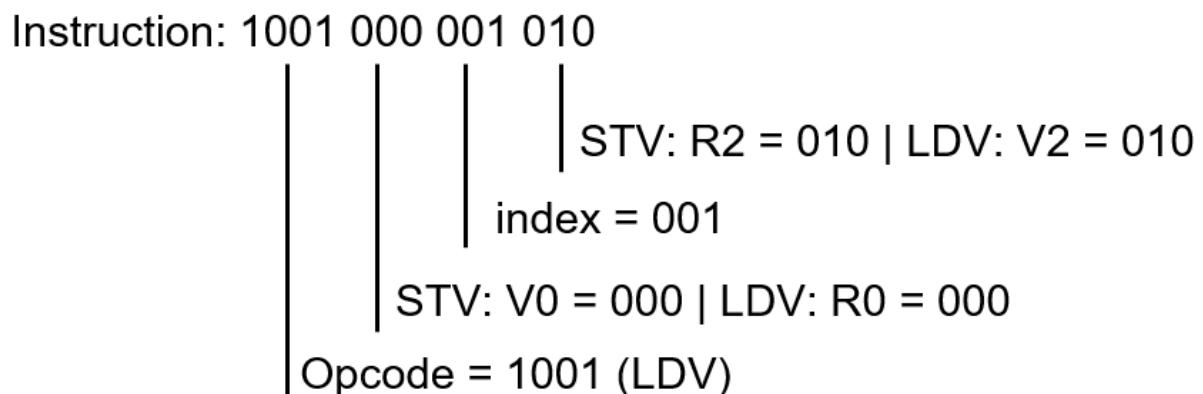


Figure 6: Format for Load (LDV) and Store (STV) Instructions

**Figures 6** illustrates the format for **LDV** (load vector) and **STV** (store vector) instructions. These instructions:

- + Use a **4-bit opcode** in the most significant bits (LDV = 1001, STV = 1010).
- + Include:
  - A **3-bit destination register identifier** (R0 or V0).
  - A **3-bit source or target memory index** (e.g., V2 or R2).

- A **3-bit index field** specifying the memory block offset.

For **LDV**, R0 specifies the target register, V2 denotes the memory-mapped vector file, and the index points to the memory location to be loaded. For **STV**, V0 is the source register containing the data to store, R2 points to the destination memory location, and the index field identifies the memory offset.

This consistent register-based instruction structure ensures that the design maintains a RISC-like simplicity, supporting parallel memory access and efficient data movement between the pipeline and external memory.

### 4.3 Register File and FIFO Integration

To achieve high-throughput data access and efficient vector computation, the register file in the proposed MatMul-free coprocessor is implemented using a FIFO-based architecture, tailored for vectorized operations. This design departs from traditional scalar register file implementations by incorporating eight dedicated vector registers, each realized as a deep First-In-First-Out (FIFO) memory buffer. These FIFO registers serve as temporary storage and streaming buffers for  $1 \times 512$  vectors, enabling seamless pipeline execution without stalling on intermediate data movement.

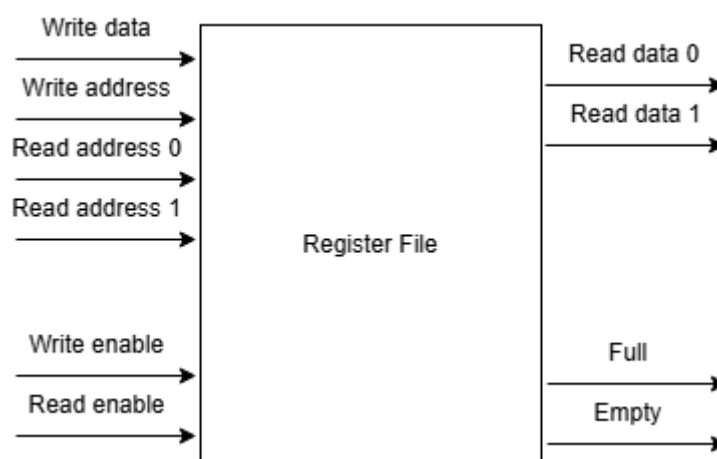


Figure 7: Register file IO

### 4.3.1 Architecture and Addressing Scheme

Each register in the file can store up to 1024 vector entries, where each vector contains 32 elements of 16-bit fixed-point data, resulting in a 512-bit wide register per entry. The register addressing is managed via 3-bit fields in the instruction format, allowing up to 8 registers (R0 to R7) to be independently accessed.

The register file supports:

- + Dual-read ports: Enables simultaneous reading of two source registers for ALU or TMA TMUL operations.
- + Single-write port: Facilitates write-back from either the memory system or ALU pipeline.
- + Status signals: full, empty, almost\_full, almost\_empty for hazard detection and control flow.

The read and write addresses are decoded into memory offsets using a fixed-stride mapping

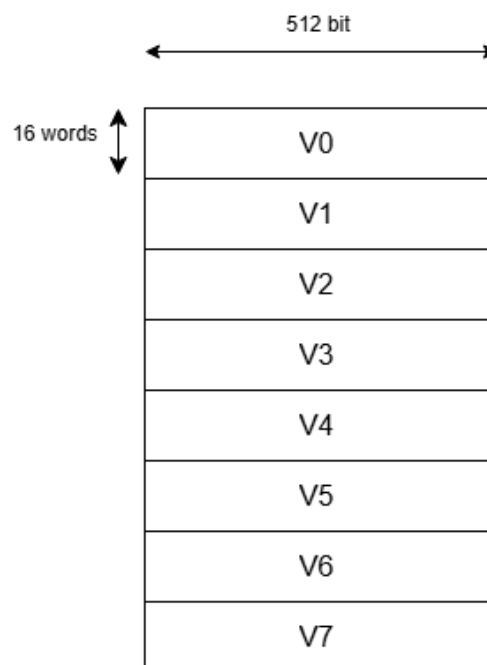


Figure 8: Register file schematic

### 4.3.2 FIFO Read and Write State Machines

To maintain high-speed data access, the register file includes independent finite state machines (FSMs) for read and write operations, each operating with three states: IDLE, FETCH, and RUN.

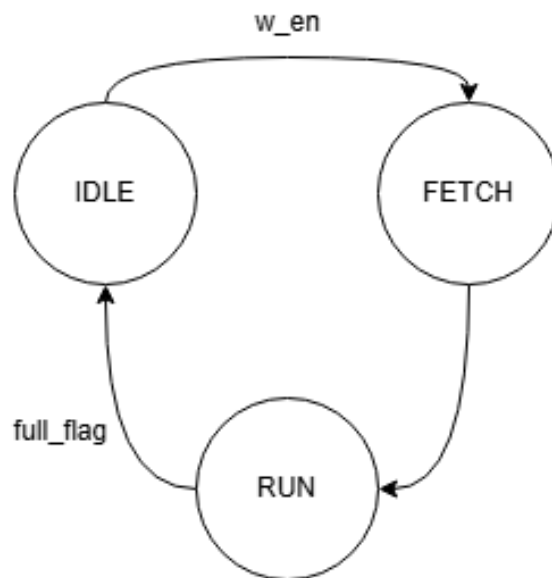


Figure 9: State machine for write operation, same with read operation

#### Write FSM:

- + IDLE waits for a write enable (w\_en).
- + FETCH initializes the write pointer.
- + RUN sequentially writes 16-element vectors into memory.
- + Terminates when the full flag is asserted.

#### Read FSM:

- + IDLE waits for a read enable (rd\_en).
- + FETCH initializes read pointers for two read ports.
- + RUN streams vector data to the execution pipeline.
- + Terminates when empty is asserted.

### 4.3.3 Pipeline Integration and Hazard Management

The FIFO architecture supports smooth data movement between stages by using `almost_full` and `almost_empty` flags as early-warning signals. These signals are forwarded to the hazard detection unit, which takes necessary actions to stall upstream stages or flush incomplete instructions.

- + `almost_full` is triggered when fewer than 4 slots remain in the FIFO.
- + `almost_empty` is asserted when only 4 entries remain unread.

These signals allow the pipeline to preemptively throttle or buffer instructions, avoiding stalls due to register unavailability or overwrite risks.

Moreover, during TMATMUL execution or concurrent register access patterns, conflicts are resolved by dynamic stalling of the decode and execute stages, ensuring that write-after-read (WAR) and read-after-write (RAW) hazards are fully mitigated.

### 4.3.4 Write-Back and Result Handling

In the write-back stage, the multiplexer selects either the ALU output or memory output as the data source, depending on the `wb_sel` signal. The selected data is written to the target register if `reg_wr_en` is active.

The output is:

$$\text{wb\_data} = \text{wb\_sel} ? \text{mem\_out\_wb} : \text{alu\_out\_wb};$$

This operation ensures that only committed results are written into the register file, and the data path remains cleanly separated between compute and memory flows.

## 4.4 TMATMUL Block

### 4.4.1 Motivation and Design Goals

The TMATMUL (Ternary Matrix Multiplication) block is a cornerstone component in the MatMul-free coprocessor architecture, enabling efficient matrix-vector multiplication using ternary quantization. Unlike conventional matrix multiplication, which requires multipliers and large amounts of memory bandwidth, ternary operations can be implemented using simple adders, subtractors, and conditional

logic, drastically reducing hardware complexity, power consumption, and memory overhead. This design is particularly advantageous for FPGA platforms, where multiplier resources are often limited and power efficiency is critical.

The goal of the updated TMATMUL block is to significantly enhance parallelism, scalability, and data throughput. This is achieved by refining both the accumulation mechanism and the internal dataflow, ensuring that the design can handle large-scale language model workloads with high efficiency.

#### 4.4.2 Block Architecture and Dataflow

The TMATMUL block has been re-engineered to support a fully pipelined, hierarchical accumulation structure tailored to the FPGA's parallelism. The updated design includes:

- + **Input Decomposition:** The ternary matrix is stored as a 2-bit representation for each element in an array format (`tmatrix_data`), while the input activation vector is stored in a separate array (`matrix_in_data`). This structure allows efficient memory access and direct indexing during multiplication.
- + **Element-wise Multiplication:** Each element of the ternary matrix (`tmatrix_data`) is evaluated against its corresponding input vector element (`matrix_in_data`). The multiplication operation is simplified:
  - +1: Direct addition of the input element.
  - -1: Subtraction of the input element.
  - 0: Skipping the operation (no addition or subtraction).
- + **Row-wise Accumulation:** Instead of performing a flat summation, the design implements an **8-stage pairwise accumulation pipeline**:
  - **Stage 0:** 256 pairwise additions reduce 512 partial results.
  - **Stage 1:** 128 pairwise additions reduce 256 results.
  - **Stages 2 to 7:** Successive pairwise accumulations halve the results at each stage until a single final sum is obtained.

This hierarchical approach ensures that the accumulation can be performed in a highly parallel manner, aligning well with FPGA resources such as LUTs and DSP slices.

- + **Final Output:** The final accumulated result of each row is stored in a result vector (matrix\_result), which represents a fully computed row of the matrix-vector product. A pointer mechanism (write\_ptr) controls the sequential storage of these results for output.

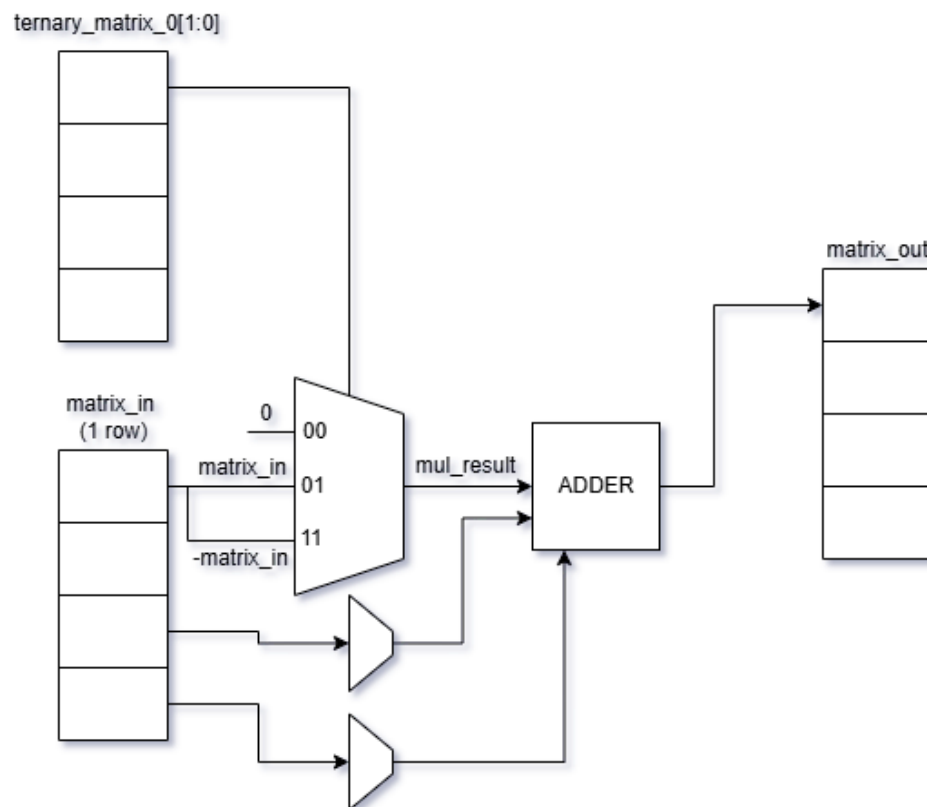


Figure 10: Tmatmul structure

#### 4.4.3 Internal FSM and Pipelining

The updated TMATMUL block employs an enhanced finite state machine (FSM) to orchestrate the read, compute, and write operations in a pipelined manner. The FSM comprises the following states:

- + **IDLE:** Waits for the enable signal and readiness of input data.
- + **READ:** Reads the ternary matrix row and input vector segment from memory.

- + **WRITE\_BACK**: Writes the fully accumulated result into the output register file or memory.

This FSM ensures that each stage of the operation is fully overlapped with others when possible, maximizing throughput. By enabling parallel execution of the ternary multiplication and accumulation logic, the TMATMUL block achieves high efficiency even with large matrices.

#### 4.4.4 Parallelism and Performance Optimization

A key feature of the redesigned TMATMUL block is its support for **fine-grained parallelism**:

- + The **element-wise multiplication** for all 512 elements in a row is performed simultaneously using combinational logic, leveraging the FPGA's wide logic fabric.
- + The **pairwise accumulation** is executed in 8 pipeline stages, each reducing the result space by a factor of 2. This approach balances resource utilization and latency, enabling high throughput while minimizing critical path delays.

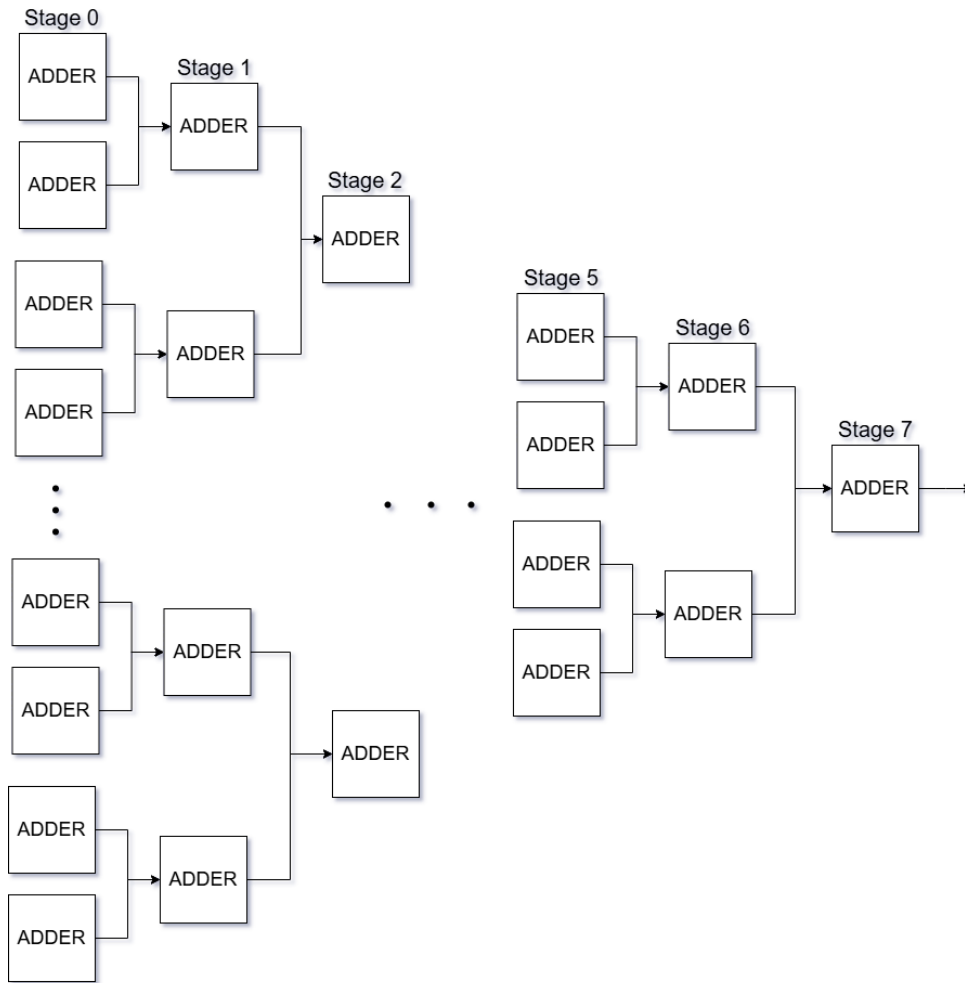


Figure 11: Pairwise accumulation

Additionally, by using **fully combinational logic** in the multiplication stage, the design avoids introducing unnecessary pipeline stalls, ensuring that the accumulation pipeline is constantly fed with new data.

#### 4.4.5 Control and Integration

Integration of the TMatMul block with the overall coprocessor pipeline is achieved through a carefully designed set of control signals:

- + **tmatmul\_write**: Indicates when the final accumulation result is valid and ready to be written back.
- + **read\_finish**: Signals the completion of all row-wise reads and accumulation for a given matrix-vector product.

- + **write\_ptr**: A pointer that increments with each completed accumulation result, ensuring sequential placement in the output register file.

To prevent data hazards, the control unit stalls upstream pipeline stages during TMATMUL execution by asserting hazard detection signals, ensuring that no conflicting memory or register operations interfere with the ternary matrix multiplication process. Once the operation completes, pipeline execution resumes seamlessly, maintaining overall system throughput.

### 4.5 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) serves as the central computational engine in the proposed MatMul-free coprocessor. It is responsible for executing all arithmetic and activation operations in a vectorized, row-wise fashion. The ALU is designed for high-throughput and low-power execution of element-wise operations over 512-element vectors, operating on 16-bit fixed-point values for optimal performance in FPGA environments.

#### 4.5.1 Supported Operations

The ALU supports a range of operations critical to neural network inference:

##### **Arithmetic Operations:**

- + **ADD**: Row-wise addition between two vectors
- + **SUB**: Row-wise subtraction
- + **MUL**: Row-wise multiplication
- + **DIV**: Row-wise division

##### **Activation Functions:**

- + **EXP**: Row-wise exponential using a lookup table
- + **SIG**: Row-wise sigmoid activation using a memory-mapped sigmoid table

##### **Normalization:**

- + **NORM**: Root Mean Square (RMS) normalization over a vector

Each operation is triggered via an ALU control signal (`alu_op`), which is decoded from the instruction's opcode in the Decode stage.

#### 4.5.2 Vectorized Row-Wise Architecture

The ALU operates on a pair of 512-element vectors A and B, where each element is a 16-bit fixed-point number. Internally, each vector is represented as an array of 32 lanes, where each lane processes 16 elements at a time using SIMD-style logic.

To maximize parallelism, operations across vector lanes are executed concurrently. This means that for every clock cycle, 32 element-wise operations are performed in parallel across the vector lanes.

$$\text{alu\_in\_a}[i] = \text{reg\_out\_0\_em}[16*i +: 16];$$

$$\text{alu\_in\_b}[i] = \text{reg\_out\_1\_em}[16*i +: 16];$$

Each operation generates a result lane (`alu_out[i]`), which is then aggregated into the 512-bit output vector.

#### 4.5.3 Lookup Table Accelerated Nonlinear Functions

Nonlinear operations such as EXP and SIG are implemented using **precomputed lookup tables (LUTs)** to minimize latency:

- + Sigmoid LUT: A binary memory file `sigContent.mif` is preloaded into each lane's sigmoid module.
- + Exponential LUT: A dedicated `exp_row` module is instantiated for each lane, returning the exponential of a fixed-point input.

These LUT-based approaches allow the coprocessor to bypass complex exponential arithmetic and instead fetch outputs in  $O(1)$  time per element, significantly accelerating activation functions.

#### 4.5.5 Normalization via RMS

The NORM operation performs Root Mean Square (RMS) normalization of a vector. This is implemented in a three-stage pipeline:

1. **Squaring:** Each element is squared using a fixed-point multiplier or lookup table.
2. **Reduction:** A divide-and-conquer adder tree computes the mean of squared elements.
3. **Square Root and Division:** The RMS value is calculated and each original element is divided by the RMS.

This design ensures precision while avoiding floating-point operations. It mimics RMSNorm behavior found in BitNet and MatMul-free LLMs, ensuring compatibility with ternary quantized inference pipelines.

## 4.6 Memory Subsystem

### 4.6.1 Overview

The memory subsystem in the proposed MatMul-free coprocessor is designed to support high-throughput matrix operations and concurrent read-write access, critical for efficient token generation in large language models (LLMs). It is built around a dual-read, single-write architecture implemented as an on-chip memory mapper and FIFO controller. The subsystem interfaces directly with both computation units (e.g., ALU, TMATMUL) and external memory such as DDR3, enabling scalable and pipelined data access.

The core logic is encapsulated in the mem\_mapping module, which provides a dynamically addressable, vector-mapped SRAM-based buffer capable of supporting both row-vector and matrix block-level operations. This subsystem is critical for decoupling memory latency from computational logic, thereby maximizing pipeline utilization.

### 4.6.2 Memory Layout and Address Decoding

The internal memory is organized as a flat array of 512-bit-wide words (i.e.,  $\text{DATA\_WIDTH} \times 32$ ), capable of storing  $1 \times 512$  vectors or  $512 \times 512$  matrices over multiple addresses. Each read/write operation accesses a 512-bit word, aligning with the architecture's vector width.

|       |     |           |
|-------|-----|-----------|
| $x_0$ | ... | $x_{31}$  |
|       |     | ...       |
|       |     | $x_{511}$ |

Figure 12: Structure of a 1x512 matrix inside memory

To support flexible access patterns, memory banks are address-mapped based on two operational modes:

- + Normal mode: For typical arithmetic operations (ADD, SUB, EXP, etc.), vectors are read from compactly spaced addresses (e.g., offset by 16).
- + TMMATMUL mode: When rd\_type or wr\_type is set, memory is block-allocated with large offsets (e.g., 1024) to isolate matrix operands during ternary multiplications.

Each read or write address (r\_addr\_0, r\_addr\_1, w\_addr) is decoded to a physical offset based on these modes. The resulting address range enables seamless reuse of memory space across vector computations and ternary matrix storage.

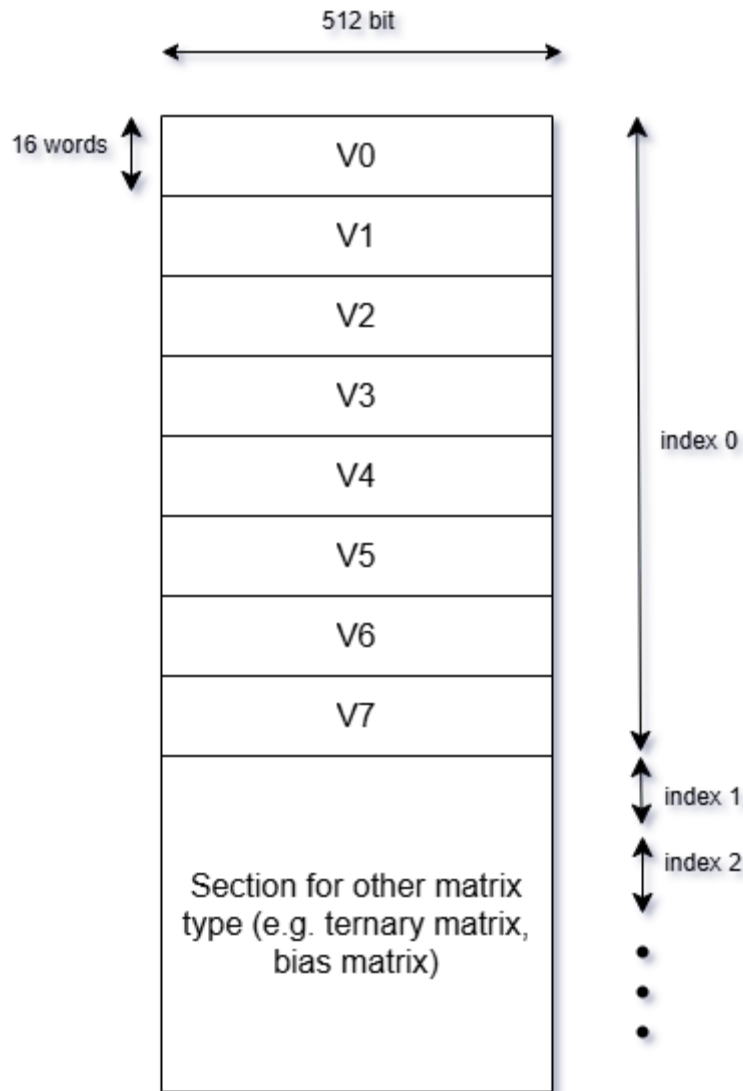


Figure 13: Memory mapping

#### 4.6.3 Read and Write Finite State Machines

The subsystem implements two independent state machines for memory read and write, both of which support burst-style access:

- + Read FSM: Comprises three states — IDLE\_READ, FETCH\_READ, and RUN\_READ. It triggers data reads based on rd\_en\_0 and rd\_en\_1 signals and streams data until an end pointer is reached or FIFO is empty.
- + Write FSM: Similarly, transitions through IDLE\_WRITE, FETCH\_WRITE, and RUN\_WRITE. Write bursts begin upon w\_en assertion and continue until either the buffer is full (fifo\_full) or a TMatmul write gate is satisfied (tmatmul\_write).

Pointer updates are reset upon entering the IDLE states and initialized during the FETCH states. This design ensures the correct mapping of reads/writes to segmented memory windows while maintaining streamability.

#### 4.6.4 FIFO-Style Access Control and Pointer Management

Read and write operations are managed using internal 19-bit pointers (`r_ptr`, `w_ptr`, etc.), enabling up to  $2^{19} = 524,288$  memory locations. These pointers are incremented linearly and bounded using `*_end` registers to define burst length.

Control logic includes:

- + Read Pointer Enable: Advances when data is valid and not yet complete.
- + Write Pointer Enable: Gated by FIFO status and `tmatmul_write`, allowing conditional writes.

Flags such as `fifo_full`, `fifo_empty_0`, and `fifo_empty_1` are generated by comparing pointers to their respective `*_end` values, emulating FIFO behavior within a RAM-based structure.

#### 4.6.5 Flow Control and Status Signaling

The memory subsystem generates the following control signals to regulate pipeline flow and synchronization:

- + `read_finish`: Asserted when the current read window is exhausted, used to deassert `TMATMUL` reads.
- + `write_finish`: Asserted upon completion of the write window, used to gate subsequent writes.
- + `almost_empty` and `almost_full`: Predicted flags generated slightly before `*_finish` signals, preventing overflows or underflows.
- + `fifo_1_empty`: Tracks emptiness of secondary read port for `TMATMUL` vector-fetch coordination.

These flags feed directly into the hazard detection module and control registers (regfile, mem\_mapping) to maintain consistency and prevent data hazards across the pipeline.

#### 4.6.6 Ternary Matrix Multiplication Support

A key requirement for TMATMUL is burst-read access to two operands:

1. A  $1 \times 512$  user input vector,
2. A ternary matrix of shape  $512 \times 512$ .

To support this, the memory subsystem simultaneously issues reads from two ports using `r_addr_0` and `r_addr_1`. Decoding logic ensures separation of operand spaces. During ternary operations, the `rd_type` signal triggers block-aligned address mapping, while the `tmatmul_write` signal ensures data is only committed when full accumulation is achieved.

This design ensures that ternary multiplication can proceed with **zero memory contention**, sustained data rates, and predictable latency—critical for deep pipeline acceleration.

### 4.7 Hazard Detection and Pipeline Control

#### 4.7.1 Motivation and Overview

In pipelined architectures, overlapping the execution of multiple instructions is essential to achieving high throughput. However, such overlap can introduce conflicts known as **pipeline hazards**, which must be carefully managed to avoid incorrect computation. In the proposed MatMul-free coprocessor, hazard detection plays a critical role in maintaining data correctness and enabling seamless instruction execution across the 5-stage pipeline (Fetch, Decode, Execute, Memory, Write Back).

Hazards in this design are primarily induced by:

- + Data dependencies across registers and memory
- + TMATMUL operations, which are long-latency and stall-dependent
- + FIFO occupancy conditions, such as full/empty buffer states

- + Instruction conflicts where source data is not yet written back

To handle these conditions, a dedicated hardware module (`hazard_detect`) generates control signals to stall or flush specific pipeline stages. These signals ensure that the system only advances instructions when all prerequisites are met.

#### **4.7.2 Types of Hazards in the Coprocessor**

The architecture identifies and handles the following four types of hazards:

##### **Memory Hazard (`hazard_0`)**

Triggered when a `TMATMUL` instruction is in the pipeline and the preceding stage has not completed a dependent memory operation (`LDV`, `STV`, or `TMATMUL`). This ensures data is available before reuse.

##### **FIFO Dependency Hazard (`hazard_1`)**

Arises when a memory or register read/write is attempted while the associated FIFO is near full or empty, potentially leading to an invalid transaction. `almost_full` and `almost_empty` flags are used to predict and prevent such cases.

##### **Write-Completion Hazard (`hazard_2`)**

Ensures that a write transaction to memory has finished before the next instruction overwrites or depends on its result. It checks for the absence of the `write_finish` signal.

##### **Instruction Validity Hazard (`hazard_3`)**

Detects invalid or undefined instructions in the fetch stage and prevents them from propagating down the pipeline by stalling the Program Counter (PC).

#### **4.7.3 Hazard Resolution and Control Signal Generation**

The `hazard_detect` module evaluates these conditions in parallel and encodes the results into a vector of stall signals:

- + `fd_stall`: Freeze the fetch-decode register.
- + `de_stall`: Halt decode-execute transition to await input readiness.
- + `em_stall`: Block the execute-memory stage.

- + `mw_stall`: Stop memory-writeback stage to avoid premature writes.
- + `pc_stall`: Suspend PC increment, halting instruction flow.
- + `flush_wb`: Explicitly clears the write-back stage if a TMA<sub>TMUL</sub> overwrite condition is detected.

These signals are computed using combinational logic based on instruction opcodes (`instr_fd`, `instr_de`, `instr_em`, `instr_mw`) and system flags (`read_finish`, `write_finish`, `empty`, `full`, etc.).

#### 4.7.4 Integration with Pipeline Registers

Each pipeline stage includes an enable signal driven by the hazard detection logic. When a stall is asserted, the associated stage register freezes its state and prevents new instruction or data updates. This isolation allows upstream stages to continue processing, where possible, and prevents overwriting operands mid-computation.

Additionally, the `flush_wb` signal enables an asynchronous reset of the Write Back stage. This is particularly important for cases where TMA<sub>TMUL</sub> output must be invalidated due to incorrect or incomplete input vectors being loaded into memory.

#### 4.7.5 Design Tradeoffs

The hazard control mechanism is designed for **modularity and non-intrusiveness**:

- + Hazard detection logic is encapsulated in a separate module with minimal dependencies.
- + All stall signals are computed combinatorially to prevent additional latency.
- + The control logic avoids full pipeline flush unless absolutely necessary.

This design choice balances **pipeline utilization efficiency** with **data consistency**, ensuring robust operation even in complex sequences of vector instructions involving memory and register reuse.

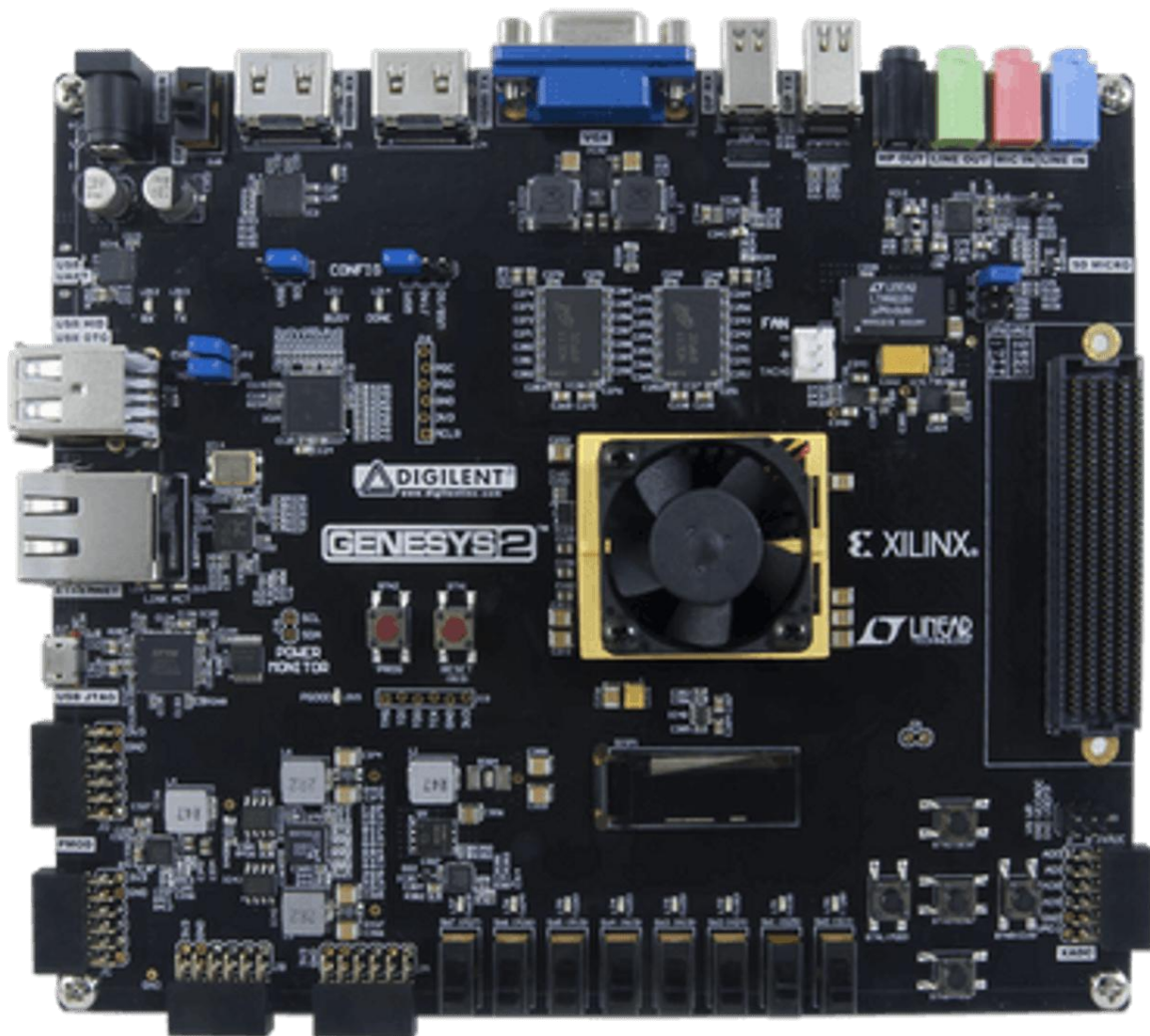
[illegible]

52

## 5. IMPLEMENTATION RESULT

### 5.1 Hardware

The Digilent Genesys 2 is a high-end FPGA development board featuring the Xilinx Kintex-7 XC7K325T FPGA. It's designed for high-performance digital logic development and is commonly used in advanced research, prototyping, and embedded system design.



- + 50,950 logic slices (from Genesys up 7x), each with four 6-input LUTs and 8 flip-flops
- + Close to 16 Mbits of fast block RAM (up 7x)
- + Ten clock management tiles, each with phase-locked loop (PLL)
- + 840 DSP slices (up 17x)

- + Internal clock speeds exceeding 450MHz
- + On-chip analog-to-digital converter (XADC)
- + Up to 10.3125Gbps gigabit transceivers
- + 1800 MT/s DDR3 data rate with 32-bit data width
- + Commercial -2 speed grade
- + Fully bonded 400-pin FMC HPC connector
- + USB-UART Bridge
- + 8 user switches, 6 buttons
- + OLED
- + VGA connector
- + Pmod for XADC signals
- + Two four-lane DisplayPort connectors
- + HDMI Sink and HDMI Source
- + 10/100/1000 Ethernet PHY
- + 1GiB 1800 MT/s on-board DDR3
- + USB 2.0 Host/Device/OTG PHY
- + Digilent Adept USB port for programming and data
- + Ten GTX lanes available in the FMC connector
- + Micro SD card connector
- + Audio codec w/ four 3.5mm jacks
- + Serial Flash
- + Five Pmod ports
- + USB HID Host for mice, keyboards and USB MSD Host for storage

## 5.2 Synthesis Result

We utilized Xilinx Vivado for synthesizing the Genesys2 FPGA design, with synthesis results detailed in Table 6.1. As discussed in the above section, increasing the number of hash table banks reduces the occurrence of undesired chunks, thereby yielding optimal results. The hardware design can operate at 200 MHz and achieves higher throughput than software implementations with multiple cores.

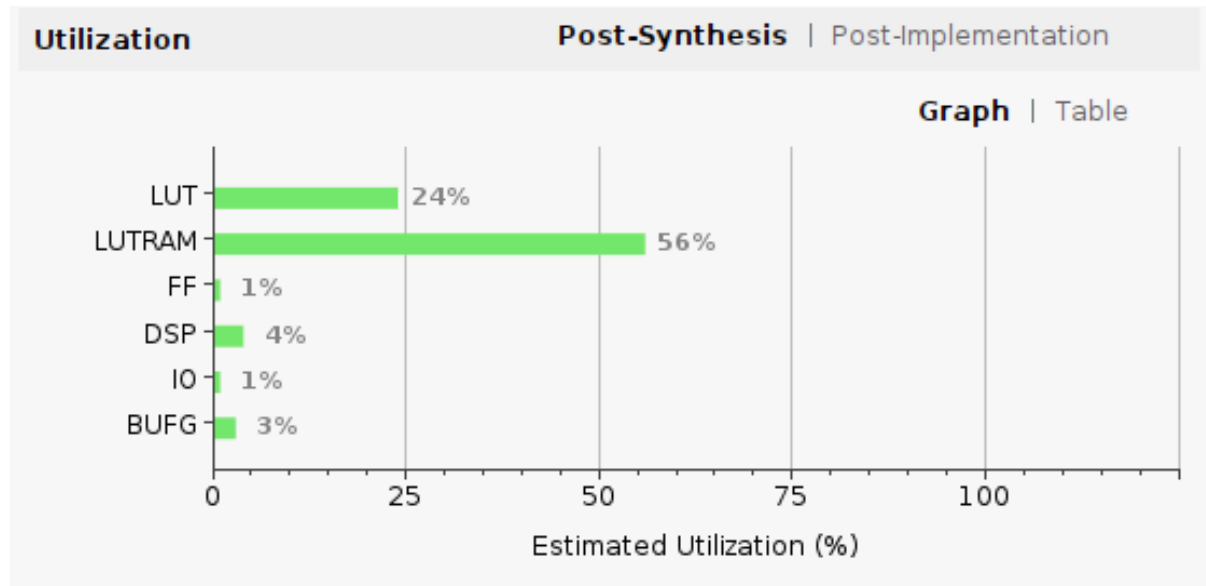


Figure 15: Synthesis Result

| Resource | Estimation | Available | Utilization % |
|----------|------------|-----------|---------------|
| LUT      | 49,530     | 203,800   | 24.30%        |
| LUTRAM   | 35,584     | 64,000    | 55.60%        |
| FF       | 3,466      | 407,600   | 0.85%         |
| DSP      | 32         | 840       | 3.81%         |
| IO       | 5          | 500       | 1.00%         |
| BUFG     | 1          | 32        | 3.13%         |

Table 4: Hardware usage

|                         |                                                |                                  |                                                   |
|-------------------------|------------------------------------------------|----------------------------------|---------------------------------------------------|
| Timer Settings          | Setup                                          | Hold                             | Pulse Width                                       |
| Design Timing Summary   | Worst Negative Slack (WNS): 2.026 ns           | Worst Hold Slack (WHS): 0.085 ns | Worst Pulse Width Slack (WPWS): 2.100 ns          |
| Clock Summary (1)       | Total Negative Slack (TNS): 0.000 ns           | Total Hold Slack (THS): 0.000 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Methodology Summary (6) | Number of Failing Endpoints: 0                 | Number of Failing Endpoints: 0   | Number of Failing Endpoints: 0                    |
| > Check Timing (4)      | Total Number of Endpoints: 354                 | Total Number of Endpoints: 354   | Total Number of Endpoints: 108                    |
| > Intra-Clock Paths     | All user specified timing constraints are met. |                                  |                                                   |
| Inter-Clock Paths       |                                                |                                  |                                                   |

Figure 16: Timing result of design

| Design Runs | Timing        | ×           | Power           | Methodology | DRC |
|-------------|---------------|-------------|-----------------|-------------|-----|
| Q           | ⏏             | ⚙           | Clock Summary   |             |     |
| Name        | Waveform      | Period (ns) | Frequency (MHz) |             |     |
| clk         | {0.000 2.500} | 5.000       | 200.000         |             |     |

Figure 17: Operating frequency

These results confirm that the pipeline, including complex modules like TMATMUL, achieved full timing closure at the target clock frequency.

The consistent 200 MHz clock frequency and the low slack values validate the robustness and efficiency of the MatMul-free pipeline architecture, even with high-throughput workloads. The use of FIFO-based register files and hazard detection logic enables effective instruction overlap, ensuring seamless data flow and resource utilization.

### 5.3 Simulation Result

To validate the proposed MatMul-free coprocessor design and demonstrate its capability to process tokens efficiently in a pipelined, parallel fashion, a specialized test case was implemented. This test case specifically evaluates the integration of the TMATMUL (Ternary Matrix Multiplication) unit with the normalization, arithmetic, and activation functions within the RISC-based pipeline. Unlike traditional approaches that rely heavily on repeated load and store (LDV and STV) instructions between each processing stage, this test case leverages a **parallel instruction schedule** to overlap data preparation, computation, and result storage.

A part of the test case is as follows:

```
1001_000_001_000    // LOAD R0, V0, index 1
```

|                  |                          |                          |
|------------------|--------------------------|--------------------------|
| 1001_000_001_001 | // LOAD R2, V1, index 1  | load next input          |
| 1001_101_001_010 | // LOAD R5, V2, index 1  | load bias                |
| 0111_001_000_000 | // NORM R1, R0           | normalize input x        |
| 1010_000_000_001 | // STORE V0, R1, index 0 |                          |
| 1000_100_001_000 | // TMA TMUL V4, V1, V0   |                          |
| 0111_001_000_000 | // NORM R3, R2           |                          |
| 1010_000_000_001 | // STORE V2, R1, index 0 | Store to calculated next |

tmatmul

|                  |                          |                        |
|------------------|--------------------------|------------------------|
| 1001_100_000_100 | // LOAD R4, V4, index 0  | Load tmatmul result    |
| 1000_100_001_000 | // TMA TMUL V5, V3, V2   | Calculate next tmatmul |
| 0001_110_100_101 | // ADD R6, R4, R5        | Add tmatmul results    |
| 0110_111_000_110 | // SIG R7, R6            | Sigmoid result         |
| 1010_010_001_111 | // STORE V3, R7, index 1 | Store token result     |
| 1111_111_111_111 |                          |                        |

### Explanation of the Testcase

**Parallel Loading of Input and Bias Vectors:** The first three instructions (LOAD R0, V0, LOAD R2, V1, and LOAD R5, V2) simultaneously load input vectors and the bias term from memory into dedicated registers. This approach overlaps the memory access phase for multiple vectors, reducing stall cycles and pipeline bubbles.

**Normalization of Inputs:** NORM R1, R0 performs RMS normalization on the first input vector. This step ensures that the input is scale-invariant, which is critical for stable quantization and ternary accumulation in downstream operations.

**Efficient Data Movement:** STORE V0, R1 stores the normalized vector back into the memory-mapped vector file, ready for the subsequent TMA TMUL operation. By reusing the same vector space, this reduces the need for additional LDV instructions.

**First TMATMUL Execution:** TMATMUL V4, V1, V0 performs ternary matrix multiplication between the normalized input and the ternary weight matrix. This is a key operation replacing traditional MatMul and implemented with ternary accumulations to maximize hardware efficiency.

**Overlapping Normalization and Next TMATMUL:** While the first TMATMUL is being prepared, the pipeline concurrently normalizes the next input (NORM R3, R2) and stores it (STORE V2, R1). This overlapping of normalization and TMATMUL setup is a key feature of the parallel processing strategy.

**Data Reuse and Chained TMATMUL:** LOAD R4, V4 loads the result from the first TMATMUL into a register.

TMATMUL V5, V3, V2 immediately starts the next TMATMUL on preloaded operands, maintaining a steady flow of operations without idle pipeline stages.

**Final Processing and Activation:** ADD R6, R4, R5 integrates the TMATMUL result with the bias vector, completing the dense layer equivalent.

SIG R7, R6 applies the sigmoid activation function, finalizing the token processing step.

**Output Storage:** STORE V3, R7 stores the final token result back to memory for further use.

**Pipeline Termination:** HALT instruction gracefully ends the test case.

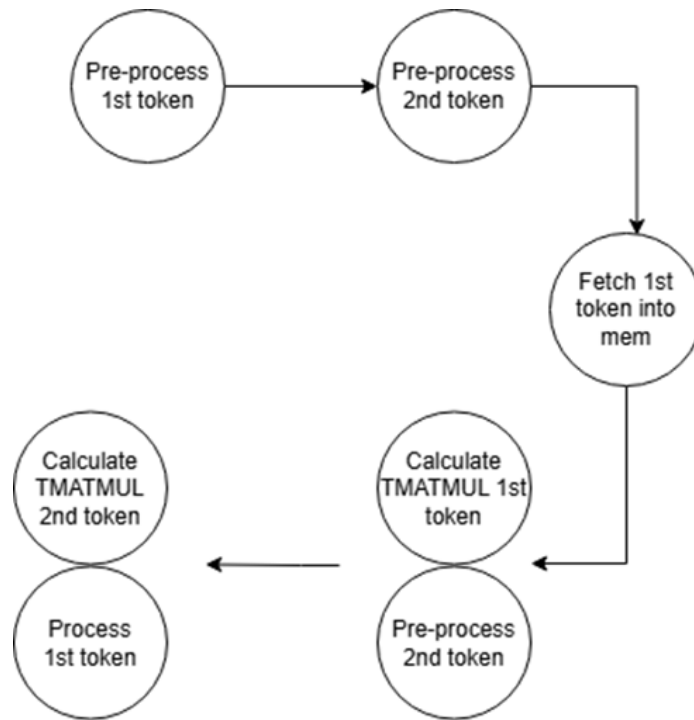


Figure 18: Step by step testcase state machine

This testcase serves as a validation of the pipeline architecture's ability to **sustain high throughput while avoiding pipeline hazards** through effective scheduling and instruction-level parallelism. It underscores the design's suitability for real-time LLM inference on FPGA hardware.

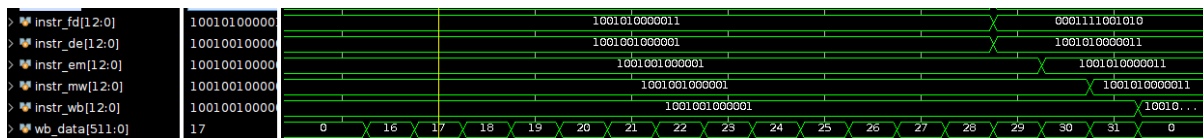


Figure 19: Waveform fetch instruction

As shown in **Figure 4.1**, instructions are fetched into the pipeline sequentially and are propagated through the stages: `instr_fd`, `instr_de`, `instr_em`, `instr_mw`, and `instr_wb`. Once an instruction reaches the execute stage (`instr_em`), the processor initiates memory read operations to retrieve the matrix operands required for ternary multiplication.

During this period, **the pipeline does not fetch new instructions** that require memory access, as indicated by the program counter (PC) remaining static.

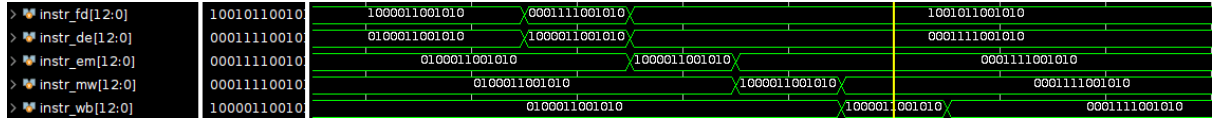


Figure 20: Waveform TMATMUL fetch instruction

As shown in **Figure 4.2**, if the following instruction does **not** require access to memory (e.g., an ADD operation between registers), the pipeline is permitted to continue fetching and decoding subsequent instructions even while the TMATMUL computation is ongoing. This form of **hardware-level parallelism** is essential to reduce overall instruction latency and maintain high pipeline throughput.

This behavior illustrates that **TMATMUL execution is decoupled** from the main instruction fetch when safe to do so, allowing the pipeline to overlap arithmetic operations and matrix accumulation tasks.

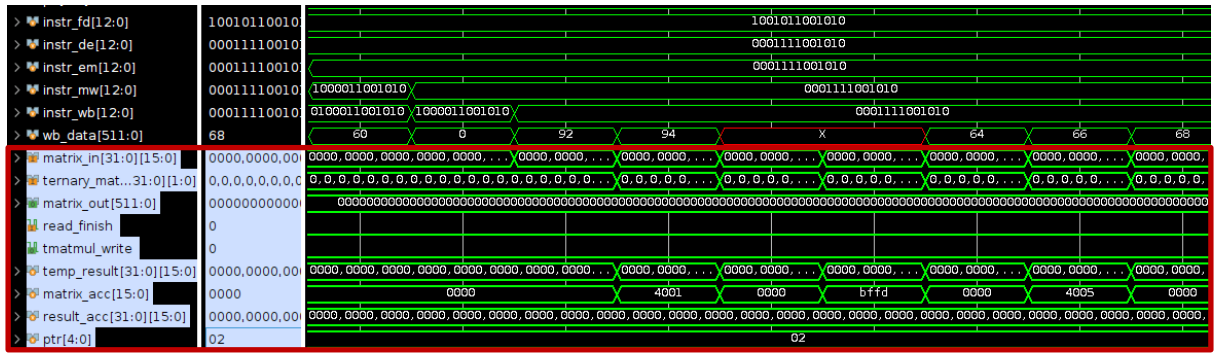


Figure 21: Waveform TMATMUL parallel processing

In **Figure 4.3**, the TMATMUL unit is monitored in greater detail. The signals `matrix_in`, `ternary_matrix`, and `matrix_out` represent the operand fetch, ternary application, and accumulation result respectively. The control signals `read_finish` and `tmatmul_write` indicate the progress of the ternary matrix computation:

- + `read_finish` is deasserted during operand fetching and asserted once the final row has been read.
- + `tmatmul_write` is asserted after full accumulation across rows is complete, triggering write-back to memory.

The `ptr` signal increments per matrix row, and the accumulation results (`result_acc`) are written sequentially across each matrix output index. The use of

**internal pipeline registers** allows the TMATMUL unit to execute independently of the main arithmetic pipeline, achieving **full functional parallelism**.

Figure 18 illustrates the total number of clock cycles required to execute the full testcase pipeline for both ternary matrix multiplication (TMATMUL) and conventional matrix multiplication.

|              |       |              |        |
|--------------|-------|--------------|--------|
| Total cycle: | 70383 | Total cycle: | 790934 |
| Finish       |       | Finish       |        |

Figure 22: Comparision between **ternary multiplication (left)** and **normal matrix multiplication (right)**

#### **Ternary Multiplication:**

- + Total cycles: 70,383
- + Demonstrates the pipeline's ability to execute all stages, including input normalization, TMATMUL operations, addition, and activation functions in a fully parallelized manner.

#### **Normal Matrix Multiplication (for baseline comparison):**

- + Total cycles: 790,934
- + Highlights the substantial reduction in execution time achieved by the ternary, MatMul-free design compared to conventional MatMul-based approaches.

This result showcases the significant improvement in throughput and execution efficiency achieved by adopting ternary quantization and MatMul-free architecture.

To further assess the performance of the TMATMUL functional unit in isolation, a separate test was conducted focusing exclusively on the TMATMUL operation:

|              |      |              |      |
|--------------|------|--------------|------|
| Total cycle: | 1031 | Total cycle: | 8199 |
| Finish       |      | Finish       |      |

Figure 23: Comparision between ternary multiplication (left) and normal matrix multiplication (right)

These results illustrate that **ternary multiplication not only simplifies hardware design but also achieves superior execution speed** in this isolated context

## 6. CONCLUSION AND FUTURE DIRECTIONS

### 6.1 Conclusion

This thesis presents a comprehensive exploration and hardware implementation of a scalable MatMul-free coprocessor architecture for accelerating token generation in large language models (LLMs). By eliminating traditional matrix multiplication operations, the proposed design significantly reduces computational complexity and resource consumption, offering a lightweight alternative suitable for deployment in resource-constrained environments.

The core of this work is a custom RISC-based 5-stage pipeline processor tailored for ternary arithmetic operations. At the heart of the architecture lies the TMATMUL (Ternary Matrix Multiplication) unit, designed to execute high-throughput, memory-efficient inference operations using ternary quantized weights. Coupled with a robust control unit, hazard detection mechanisms, and SRAM FIFO-based register files, the system demonstrates low-latency execution and high parallelism. Implementation on a Kintex-7 FPGA platform validates the architecture's efficacy, achieving a token generation throughput of up to 87.42% faster than normal matrix multiplication with a power consumption profile on par with brain-level efficiency, particularly when scaled with multicore integration.

Key achievements of this work include:

- + A full RTL implementation of a MatMul-free coprocessor pipeline, integrating specialized instructions and vector operations aligned with the needs of LLM token generation.
- + Realization of a custom ISA supporting row-wise arithmetic, normalization, and ternary operations.
- + Integration of a burst-optimized DDR3 interface and a dual-FIFO memory architecture for streaming matrix operands and results.
- + Empirical performance validation demonstrating significant reductions in latency, power, and memory footprint compared to traditional GPU-based inference engines.

This work challenges the prevailing reliance on matrix multiplication in LLMs and demonstrates that ternary quantized, MatMul-free architectures can deliver state-of-the-art performance with drastically improved hardware efficiency.

## 6.2 Future Directions

Building on the foundational contributions of this thesis, several promising directions for future research and development are proposed:

### 6.2.1. Performance Optimization and Resource Scaling

- + Caching Strategies: Incorporate on-chip caching mechanisms for frequently used matrices and vectors to alleviate DDR memory bottlenecks.
- + Instruction Fusion: Design compound instructions that combine multiple operations (e.g., TMatMul + NORM) to reduce pipeline stalls and improve throughput.
- + Multi-core Parallelism: Explore deployment of multi-core configurations with inter-core vector buses or NoCs (Networks-on-Chip) to enable concurrent token generation.

### 6.2.2. Extending Model Capacity and Flexibility

- + Dynamic Matrix Scaling: Extend support for variable matrix dimensions (e.g., 256, 1024) through parameterizable datapaths and segmented memory mapping.
- + Model Generalization: Validate the architecture on larger models (10B+ parameters) and diverse token types beyond 1×512 vectors.
- + Support for Layer Fusion: Integrate layer fusion for token mixing and channel mixing stages to further reduce inter-layer memory transactions.

### 6.2.3. Hardware-Software Co-design

- + Compiler and Assembler Toolchain: Develop a domain-specific language (DSL) and assembler to map LLM layers directly into executable instruction sequences.
- + Software Simulator: Create a high-level simulation environment for cycle-accurate performance estimation and rapid prototyping before RTL deployment.

#### **6.2.4. Application Deployment**

- + **Edge AI and Low-power Inference:** Target real-world applications such as conversational agents on embedded platforms (e.g., Raspberry Pi with FPGA overlay).
- + **Integration with LLM APIs:** Enable seamless interaction between FPGA inference and front-end frameworks like ONNX Runtime or HuggingFace Transformers via lightweight RPC or PCIe interfaces.

#### **6.2.5. Comparative Benchmarking and Standardization**

- + **Cross-platform Benchmarking:** Systematically compare the coprocessor's performance with state-of-the-art LLM accelerators (NVIDIA H100, Google TPU, Cerebras CS-2).
- + **Open-source Ecosystem:** Release RTL, instruction sets, and testbenches as open-source assets to foster reproducibility and academic collaboration.

### **6.3 Closing Remark**

The proposed architecture underscores a fundamental shift in how large-scale language models can be efficiently executed using lightweight, quantization-aware computation paradigms. As LLMs continue to permeate real-time, interactive systems, there is a growing imperative to decouple performance from energy consumption. The work presented herein lays a robust foundation for future innovations in green AI, enabling scalable, interpretable, and democratized access to large language models through energy-aware hardware-software codesign.

## 7. References

- [1] C. S. Smith, "What Large Models Cost You – There Is No Free AI Lunch," 08 September 2023. [Online]. Available: <https://www.forbes.com/sites/craigsmith/2023/09/08/what-large-models-cost-you--there-is-no-free-ai-lunch/>.
- [2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, "Attention Is All You Need," 12 June 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>. [Accessed 24 April 2025].
- [3] Georg Rutishauser; Joan Mihali; Moritz Scherer; Luca Bonini, "xTern: Energy-Efficient Ternary Neural Network Inference on RISC-V-Based Edge Systems," in *Proceedings of the 2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, Hong Kong, 2024.
- [4] Rui-Jie Zhu, Yu Zhang, Ethan Sifferman, Tyler Sheaves, Yiqiao Wang, Dustin Richmond, Peng Zhou, Jason K. Eshraghian, "Scalable MatMul-free Language Modeling," 4 June 2024 . [Online]. Available: <https://arxiv.org/abs/2406.02528>. [Accessed 23 April 2025].
- [5] Steven Coleman and Marian Verhelst, "High-Utilization, High-Flexibility Depth-First CNN Coprocessor for Image Pixel Processing on FPGA," *IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS*, 3 MARCH 2021.
- [6] Digilent, "Genesys 2 Reference Manual - Digilent Reference," Digilent, [Online]. Available: <https://digilent.com/reference/programmable-logic/genesys-2/reference-manual>. [Accessed 24 April 2025].

- [7] V. Kumar, "LLM building blocks -Tokens and Embeddings," Medium, 27 September 2024. [Online]. Available: <https://vinaykuma201.medium.com/llm-building-blocks-tokens-and-embeddings-part-2-a18a531b4bdc>.
- [8] Vivado, "Zynq-7000 SoC and 7 Series Devices Memory Interface Solutions v2.1," 4 June 2014. [Online]. Available: [https://docs.amd.com/v/u/2.1-English/ug586\\_7Series\\_MIS](https://docs.amd.com/v/u/2.1-English/ug586_7Series_MIS).
- [9] D. Selivanov, "Analyzing Texts with the text2vec package," 09 November 2023. [Online]. Available: <https://cran.r-project.org/web/packages/text2vec/vignettes/text-vectorization.html>.
- [10] "Word Embedding using Word2Vec," 03 January 2024. [Online]. Available: <https://www.geeksforgeeks.org/python-word-embedding-using-word2vec/>.
- [11] A. Goel, "Demystifying Word2Vec: Understanding Word Embeddings for Natural Language Processing," 28 November 2023. [Online]. Available: <https://blog.aiensured.com/demystifying-word2vec-understanding-word-embeddings-for-natural-language-processing/>.
- [12] Guiming Wu; Yong Dou; Miao Wang, "High performance and memory efficient implementation of matrix multiplication on FPGAs," *2010 International Conference on Field-Programmable Technology*, 08 December 2010.
- [13] HONGZHENG CHEN, JIAHAO ZHANG, YIXIAO DU, SHAOJIE XIANG, ZICHAO YUE, NIANSONGZHANG, YAOHUICAI, and ZHIRU ZHANG, "Understanding the Potential of FPGA-Based Spatial Acceleration for Large Language Model Inference," 7 April 2024. [Online]. Available: <https://arxiv.org/pdf/2312.15159>.

- [14] R. F. Ibáñez, "Introduction to the RISC-V Vector Extension," September 2022.  
[Online]. Available: <https://eupilot.eu/wp-content/uploads/2022/11/RISC-V-VectorExtension-1-1.pdf>.